A.B.V.M. Agrawal Jatiya Kosh's
Ghanshyamdas Jalan College of Science & Commerce
Malad (East), Mumbai
(Affiliated to University of Mumbai)

DEPARTMENT OF INFORMATION TECHNOLOGY

Android Practical

B.Sc. Information Technology

Submitted by

Ayesha Sarguru

Seat Number: 502520019

Under the Guidance of

Prof. Gufran Qureshi

Academic Year 2026–27

October 2026



Ghanshyamdas Jalan College of Science & Commerce

Malad (East), Mumbai

(Affiliated to University of Mumbai)

Certificate by the Institute

Department of Information Technology

I hereby certify that Ms. **Ayesha Sarguru**, Roll Number 20, a student of Ghanshyamdas Jalan College of Science & Commerce, Malad (East), studying in **B.Sc. Information Technology**, has successfully completed the practical work in **Android** for the academic year 2026–27, under the guidance of Prof. Gufran Qureshi. To the best of my knowledge, the work of the student is original and the information included in the practical is correct.

Internal Guide	**External Examiner**	**Head of the Department**

Date: _____ **College Seal:**

INDEX

Sr. No.	Title	Page No.
1	Develop a simple “Hello World” Android application demonstrating project creation	1
2	Develop an Android application demonstrating the use of Programming Resources (colors.xml, themes.xml, strings.xml, dimens.xml, drawable resources) and apply them in UI components.	2
3	Develop an Android application demonstrating complete Activity Lifecycle methods and analyze execution using Logcat.	4
4	Develop an Android application to create and integrate a Fragment and demonstrate Fragment lifecycle methods.	6
5	Develop an Android application implementing Fragment-based UI navigation between multiple screens.	8
6	Develop an Android application demonstrating communication between Activity and Fragment.	11
7	Develop an Android application using LinearLayout (Horizontal and Vertical) to design structured UI.	13
8	Develop an Android application using RelativeLayout to align UI components dynamically.	15
9	Develop an Android application using TableLayout to design a structured Login/Registration Form.	17
10	Develop an Android application using ScrollView to display long scrollable content.	20
11	Develop an Android application using AbsoluteLayout demonstrating fixed positioning.	22
12	Develop an Android application using FrameLayout to overlay UI elements.	24
13	Develop an Android application using ListView with ArrayAdapter and implement item click handling.	26
14	Develop an Android application using GridView with Custom Adapter and image selection handling.	28
15	Develop an Android application implementing RecyclerView as a modern alternative to ListView.	31
MODULE 2		
16	Develop an Android application implementing AppBar (Toolbar) and Options Menu.	33
17	Develop an Android application demonstrating AlertDialog and custom DialogFragment.	35
18	Develop an Android application implementing contextual menus and popup menus.	37
19	Develop an Android application implementing Explicit and Implicit Intents.	40

Sr. No.	Title	Page No.
20	Develop an Android application demonstrating Event Handling using Listeners (OnClick, OnLongClick, etc.).	42
21	Develop an Android application demonstrating Adapters to populate Spinner or ListView dynamically.	44
22	Develop an Android application implementing Started and Bound Services.	46
23	Develop an Android application implementing Notifications using Notification Manager.	49
24	Develop an Android application implementing Static and Dynamic Broadcast Receivers.	52
25	Develop an Android application implementing SQLite database CRUD operations.	55
26	Develop an Android application integrating SQLite with ListView or RecyclerView.	59
27	Develop an Android application to fetch and parse JSON data from a web service and display it in UI components.	62
28	Develop an Android application demonstrating Threads and Handlers for background processing.	65
29	Develop an Android application implementing Media APIs and Telephone APIs.	67
30	Develop an Android application implementing Runtime Permissions and security handling in AndroidManifest.xml.	70

Practical 1

Aim: Develop a simple “Hello World” Android application demonstrating project creation

An Android application is developed using Android Studio, which provides tools to create, build, and run Android projects. An Activity represents a screen of the application, while the XML layout defines its user interface. A TextView is used to display the message “Hello World” on the screen.

activity_main.xml

```
<?xml version="1.0" encoding="utf-8"?>
<androidx.constraintlayout.widget.ConstraintLayout xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    xmlns:tools="http://schemas.android.com/tools"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    tools:context=".MainActivity">

    <TextView
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Hello Ayesha!"
        app:layout_constraintBottom_toBottomOf="parent"
        app:layout_constraintEnd_toEndOf="parent"
        app:layout_constraintStart_toStartOf="parent"
        app:layout_constraintTop_toTopOf="parent" />

</androidx.constraintlayout.widget.ConstraintLayout>
```

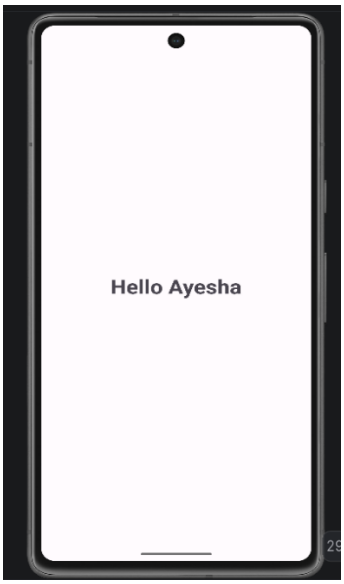
MainActivity.kt

```
package com.example.practical1

import androidx.appcompat.app.AppCompatActivity
import android.os.Bundle

class MainActivity : AppCompatActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContentView(R.layout.activity_main)
    }
}
```

Output:



Practical 2

Aim: Develop an Android application demonstrating the use of Programming Resources (colors.xml, themes.xml, strings.xml, dimens.xml, drawable resources) and apply them in UI components.

Android resources store reusable values and assets separately from application code. colors.xml defines colours, strings.xml stores text, dimens.xml stores dimensions, themes.xml defines the application theme, and drawable resources store images or drawable shapes. Using resources makes an application easier to maintain and customise.

activity_main.xml

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    xmlns:tools="http://schemas.android.com/tools"
    android:layout width="match parent"
    android:layout height="match parent"
    android:orientation="vertical"
    tools:context=".MainActivity">

    <TextView
        android:layout width="wrap content"
        android:layout_height="wrap_content"
        android:layout_gravity="center"
        android:text="Hello Ayesha!"
        android:textColor="#0B3BB3"
        android:textSize="34sp" />

    <ImageButton
        android:id="@+id/imageButton"
        android:layout_width="100dp"
        android:layout height="80dp"
        app:srcCompat="@android:drawable/btn star big on" />

    <ImageView
        android:id="@+id/imageView"
        android:layout_width="match_parent"
        android:layout_height="448dp"
        app:srcCompat="@drawable/img" />

</LinearLayout>
```

MainActvity.kt

```
package com.example.pract1

import androidx.appcompat.app.AppCompatActivity
import android.os.Bundle

class MainActivity : AppCompatActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContentView(R.layout.activity_main)
    }
}
```

colors.xml

```
<?xml version="1.0" encoding="utf-8"?>
<resources>
    <color name="colorPrimary">#008577</color>
    <color name="colorPrimaryDark">#00574B</color>
    <color name="colorAccent">#D81B60</color>

</resources>
```

theme.xml

```
<resources xmlns:tools="http://schemas.android.com/tools">
```

```
    <!-- Base application theme. -->
    <style name="Base.Theme.Pract1" parent="Theme.AppCompat.Light.DarkActionBar">
        <!-- Customize your light theme here. -->
        <!-- <item name="colorPrimary">@color/my_light_primary</item> -->
        <item name="colorPrimary">@color/colorPrimary</item>
        <item name="colorPrimaryDark">@color/colorPrimaryDark</item>
        <item name="colorAccent">@color/colorAccent</item>

    </style>

    <style name="Theme.Pract1" parent="Base.Theme.Pract1" />
</resources>
```

string.xml

```
<resources>
    <string name="app_name">Practical2</string>
    <string name="full_name">Ayesha Sarguru </string>
    <string name = "roll_no">20</string>
</resources>
```

Output:



Practical 3

Aim: Develop an Android application demonstrating complete Activity Lifecycle methods and analyze execution using Logcat.

An Activity passes through lifecycle methods as it is created, displayed, paused, stopped, restarted, or destroyed. Important methods include onCreate(), onStart(), onResume(), onPause(), onStop(), onRestart(), and onDestroy(). Logcat displays application logs and helps developers observe lifecycle execution and debug errors.

activity_main.xml

```
<?xml version="1.0" encoding="utf-8"?>
<androidx.constraintlayout.widget.ConstraintLayout xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    xmlns:tools="http://schemas.android.com/tools"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    tools:context=".MainActivity">

    <TextView
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Hello Ayesha!"
        android:textSize="20sp"
        android:textStyle="bold"
        app:layout_constraintBottom_toBottomOf="parent"
        app:layout_constraintEnd_toEndOf="parent"
        app:layout_constraintStart_toStartOf="parent"
        app:layout_constraintTop_toTopOf="parent" />

</androidx.constraintlayout.widget.ConstraintLayout>
```

MainActivity.kt

```
package com.example.pract3

import androidx.appcompat.app.AppCompatActivity
import android.os.Bundle
import android.util.Log

class MainActivity : AppCompatActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContentView(R.layout.activity_main)
        print("Oncreate")
    }

    override fun onStart() {
        super.onStart()
        print("onStart")
    }

    override fun onResume() {
        super.onResume()
        print("onResume")
    }

    override fun onPause() {
        super.onPause()
        print("onPause")
    }

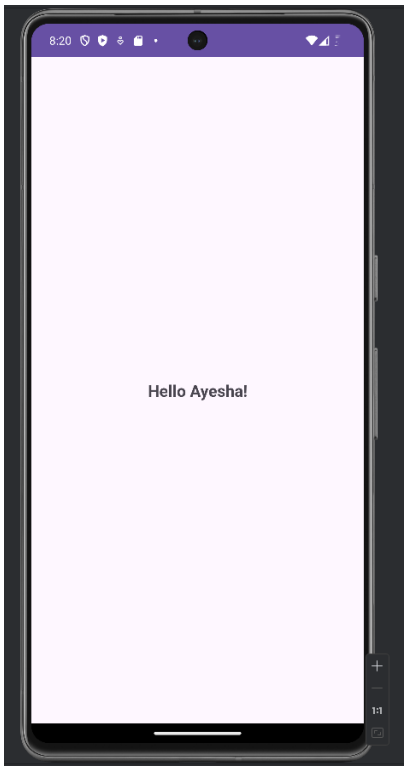
    override fun onRestart() {
        super.onRestart()
        print("onRestart")
    }

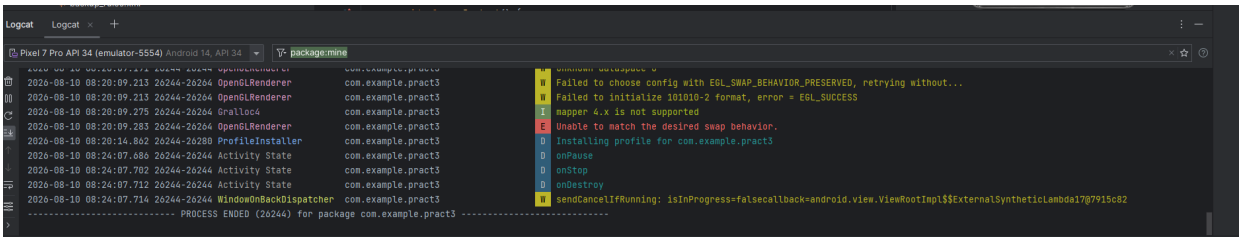
    override fun onStop() {
        super.onStop()
        print("onStop")
    }

    override fun onDestroy() {
        super.onDestroy()
        print("onDestroy")
    }
```

```
fun print(msg:String)
{
    Log.d("Activity State",msg)
}
}
```

Output





Practical 4

Aim: Develop an Android application to create and integrate a Fragment and demonstrate Fragment lifecycle methods.

A Fragment is a reusable portion of an application's user interface that is hosted inside an Activity. It has lifecycle callbacks such as onCreate(), onCreateView(), onStart(), onResume(), onPause(), onStop(), onDestroyView(), and onDestroy(). Fragments help divide a screen into modular and manageable sections.

activity_main.xml

```
<?xml version="1.0" encoding="utf-8"?>
<FrameLayout xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    xmlns:tools="http://schemas.android.com/tools"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    tools:context=".MainActivity">

    <androidx.fragment.app.FragmentContainerView
        android:id="@+id/fragmentContainerView"
        android:name="com.example.pract4.FirstFragment"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content" />
</FrameLayout>
```

MainActivity.kt

```
package com.example.pract4

import androidx.appcompat.app.AppCompatActivity
import android.os.Bundle

class MainActivity : AppCompatActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContentView(R.layout.activity_main)
    }
}
```

first_fragment.xml

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:tools="http://schemas.android.com/tools"
    android:layout width="match parent"
    android:layout_height="match_parent"
    android:gravity="center"
    android:orientation="vertical"
    tools:context=".FirstFragment">

    <!-- TODO: Update blank fragment layout -->
    <TextView
        android:id="@+id/textView"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:gravity="center"
        android:text="Ayesha Sarguru"
        android:textColor="#3F51B5"
        android:textSize="24sp"
        android:textStyle="bold|italic" />

    <TextView
        android:id="@+id/FirstFragment"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:gravity="center"
        android:text="First Fragment"
        android:textColor="#FFC107"
        android:textSize="24sp"
        android:textStyle="bold" />
```

```
</LinearLayout>
```

FirstFragment.kt

```
package com.example.pract4

import android.os.Bundle
import android.util.Log
import androidx.fragment.app.Fragment
import android.view.LayoutInflater
import android.view.View
import android.view.ViewGroup

class FirstFragment : Fragment() {
    // TODO: Rename and change types of parameters
    private val TAG = "FragmentLife"

    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        Log.d(TAG, "OnCreate")
        }


    override fun onCreateView(
        inflater: LayoutInflater, container: ViewGroup?,
        savedInstanceState: Bundle?
    ): View? {
        Log.d(TAG, "OnCreateView")
        return inflater.inflate(R.layout.fragment_first, container, false)
    }

    override fun onStart(){
        super.onStart()
        Log.d(TAG, "onStart")
    }
    override fun onResume(){
        super.onResume()
        Log.d(TAG, "onResume")
    }
    override fun onPause(){
        super.onPause()
        Log.d(TAG, "onPause")
    }
    override fun onStop(){
        super.onStop()
        Log.d(TAG, "onStop")
    }
    override fun onDestroyView(){
        super.onDestroyView()
        Log.d(TAG, "onDestroyView")
    }
    override fun onDestroy(){
        super.onDestroy()
        Log.d(TAG, "onDestroy")
    }
}
```

Output:

```
2026-07-24 15:59:12.997 11660-11660 FragmentLife           com.example.pract4                   D  OnCreate
2026-07-24 15:59:13.041 11660-11660 FragmentLife           com.example.pract4                   D  OnCreateView
2026-07-24 15:59:13.075 11660-11660 FragmentLife           com.example.pract4                   D  onStart
2026-07-24 15:59:13.081 11660-11660 FragmentLife           com.example.pract4                   D  onResume
2026-07-24 16:00:11.262 11660-11660 FragmentLife           com.example.pract4                   D  onPause
2026-07-24 16:00:11.271 11660-11660 FragmentLife           com.example.pract4                   D  onStop
2026-07-24 16:00:11.311 11660-11660 FragmentLife           com.example.pract4                   D  onDestroyView
2026-07-24 16:00:11.328 11660-11660 FragmentLife           com.example.pract4                   D  onDestroy
--------------------------- PROCESS ENDED (11660) for package com.example.pract4 ---------------------------
```

Practical 5

Aim: Develop an Android application implementing Fragment-based UI navigation between multiple screens.

Fragment-based navigation allows an application to move between different screens without creating a separate Activity for every screen. FragmentManager and FragmentTransaction are used to add, replace, or remove fragments. The back stack can preserve navigation history and allow users to return to the previous fragment.

activity_main.xml

```
<?xml version="1.0" encoding="utf-8"?>
<FrameLayout xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    xmlns:tools="http://schemas.android.com/tools"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    tools:context=".MainActivity">

    <androidx.fragment.app.FragmentContainerView
        android:id="@+id/fragmentContainerView2"
        android:name="com.example.practical5.FirstFragment"
        android:layout_width="match_parent"
        android:layout_height="match_parent" />
</FrameLayout>
```

MainActivity.kt

```
package com.example.practical5

import androidx.appcompat.app.AppCompatActivity
import android.os.Bundle

class MainActivity : AppCompatActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContentView(R.layout.activity_main)
    }
}
```

first_fragment.xml

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:tools="http://schemas.android.com/tools"
    android:layout width="match parent"
    android:layout_height="match_parent"
    android:gravity="center"
    android:orientation="vertical"
    tools:context=".FirstFragment">

    <!-- TODO: Update blank fragment layout -->
    <TextView
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="First Screen"
        android:textColor="#081F75"
        android:textSize="24sp" />

    <TextView
        android:id="@+id/textView"
        android:layout width="wrap content"
        android:layout_height="wrap_content"
        android:gravity="center"
        android:text="Ayesha Sarguru"
        android:textColor="#FF9800"
        android:textSize="24sp"
        android:textStyle="bold" />

    <Button
        android:id="@+id/btnNext"
        android:layout_width="wrap_content"
```

```
        android:layout_height="wrap_content"
        android:text="Next"
        android:textSize="24sp"
        android:textStyle="bold" />

</LinearLayout>
```

FirstFragment.kt

```
package com.example.practical5

import android.os.Bundle
import androidx.fragment.app.Fragment
import android.view.*
import android.widget.Button

//
class FirstFragment : Fragment() {


    override fun onCreateView(
        inflater: LayoutInflater, container: ViewGroup?,
        savedInstanceState: Bundle?
    ): View? {
        // Inflate the layout for this fragment
        val view = inflater.inflate(R.layout.fragment_first, container, false)
        view.findViewById<Button>(R.id.btnNext).setOnClickListener{
            parentFragmentManager.beginTransaction()
                .replace(R.id.fragmentContainerView2, SecondFragment())
                .addToBackStack(null)
                .commit()
        }
        return view
    }


}
```

second_fragment.xml

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:tools="http://schemas.android.com/tools"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:gravity="center"
    android:orientation="vertical"
    tools:context=".SecondFragment">

    <!-- TODO: Update blank fragment layout -->
    <TextView
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:gravity="center"
        android:text="Second Screen"
        android:textColor="#081F75"
        android:textSize="24sp" />

    <TextView
        android:id="@+id/textView2"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:gravity="center"
        android:text="Ayesha Sarguru"
        android:textColor="#FF9800"
        android:textSize="24sp"
        android:textStyle="bold" />

    <Button
        android:id="@+id/btnBack"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Back"
        android:textSize="24sp"
        android:textStyle="bold" />

</LinearLayout>
```

SecondFragment.kt

```
package com.example.practical5

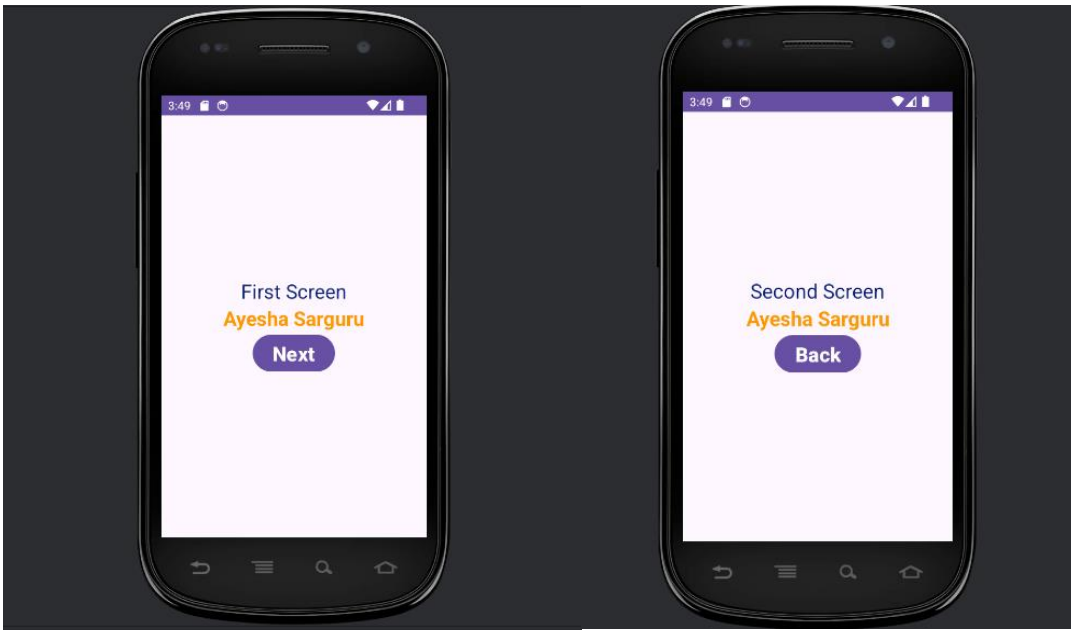
import android.os.Bundle
import androidx.fragment.app.Fragment
import android.view.*
import android.widget.Button


class SecondFragment : Fragment() {


    override fun onCreateView(
        inflater: LayoutInflater, container: ViewGroup?,
        savedInstanceState: Bundle?
    ): View? {
        // Inflate the layout for this fragment
        val view = inflater.inflate(R.layout.fragment_second, container, false)
            view.findViewById<Button>(R.id.btnBack).setOnClickListener{
                parentFragmentManager.popBackStack()

            }
        return view
        }
    }
```

Output:



Practical 6

Aim: Develop an Android application demonstrating communication between Activity and Fragment.

An Activity can host one or more Fragments, and they may need to exchange information or trigger actions. A Fragment can communicate with its host Activity through a method or a suitable communication interface. This allows UI components to work together while keeping the application organised.

activity_main.xml

```
<?xml version="1.0" encoding="utf-8"?>
<FrameLayout xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    xmlns:tools="http://schemas.android.com/tools"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    tools:context=".MainActivity">

    <androidx.fragment.app.FragmentContainerView
        android:id="@+id/fragmentContainerView2"
        android:name="com.example.practical6.FirstFragment"
        android:layout_width="match_parent"
        android:layout_height="match_parent" />

</FrameLayout>
```

MainActivity.kt

```
package com.example.practical6

import androidx.appcompat.app.AppCompatActivity
import android.os.Bundle

class MainActivity : AppCompatActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContentView(R.layout.activity_main)
    }
    fun showMessage(msg: String){
        println(msg)

    }
}
```

first_activity.xml

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:tools="http://schemas.android.com/tools"
    android:layout width="match parent"
    android:layout_height="match_parent"
    android:gravity="center"
    android:orientation="vertical"
    tools:context=".FirstFragment">

    <!-- TODO: Update blank fragment layout -->
    <TextView
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:gravity="center"
        android:text="Send Message To Main Activity"
        android:textColor="#FF9800"
        android:textSize="24sp"
        android:textStyle="bold" />

    <TextView
        android:id="@+id/textView"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:gravity="center"
        android:text="Ayesha Sarguru"
```

```
        android:textColor="#03A9F4"
        android:textSize="24sp"
        android:textStyle="bold|italic" />

    <Button
        android:id="@+id/btnNext"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Button"
        android:textSize="24sp" />

</LinearLayout>
```

FirstFragment.kt

```
package com.example.practical6

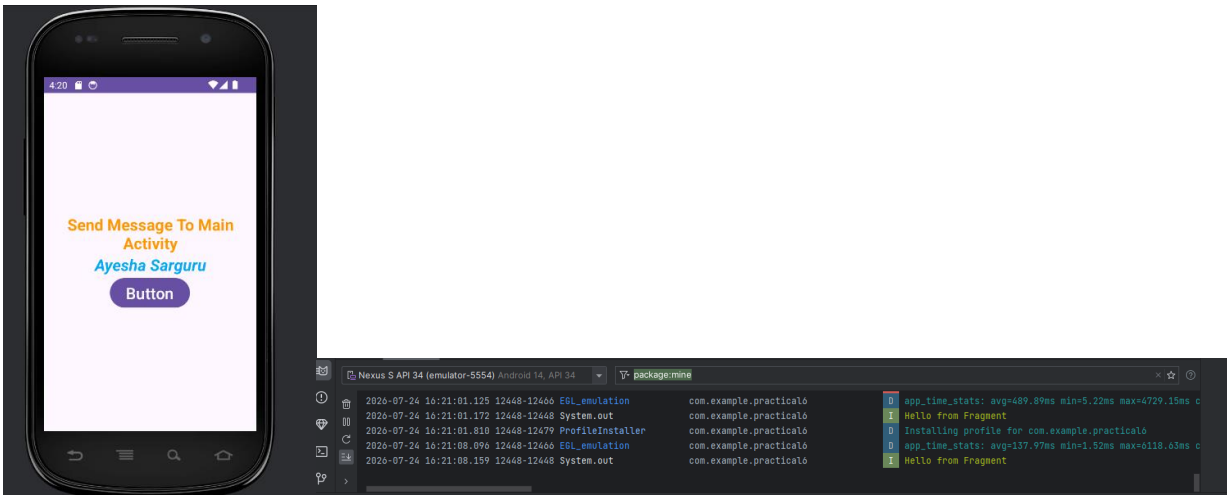
import android.os.Bundle
import androidx.fragment.app.Fragment
import android.view.*
import android.widget.Button


class FirstFragment : Fragment() {


    override fun onCreateView(
        inflater: LayoutInflater, container: ViewGroup?,
        savedInstanceState: Bundle?
    ): View? {
        // Inflate the layout for this fragment
        val view =  inflater.inflate(R.layout.fragment_first, container, false)
        view.findViewById<Button>(R.id.btnNext).setOnClickListener{
            (activity as MainActivity).showMessage("Hello from Fragment")
        }
        return view
    }


}
```

Output:



Practical 7

Aim: Develop an Android application using LinearLayout (Horizontal and Vertical) to design structured UI.

LinearLayout arranges its child views in a single row or a single column. The orientation attribute sets the arrangement to horizontal or vertical. It is useful for creating structured forms and screens containing TextViews, EditTexts, and Buttons.

activity_main.xml

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    xmlns:tools="http://schemas.android.com/tools"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:gravity="center"
    android:orientation="vertical"
    tools:context=".MainActivity">

    <TextView
        android:id="@+id/textView2"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Ayesha Sarguru"
        android:textColor="#3F51B5"
        android:textSize="24sp"
        android:textStyle="bold|italic" />

    <TextView
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Enter Your Name"
        android:textColor="#FF9800"
        android:textSize="24sp"
        android:textStyle="bold"
        tools:ignore="TextContrastCheck" />

    <EditText
        android:id="@+id/txtName"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:ems="10"
        android:inputType="text"
        tools:ignore="SpeakableTextPresentCheck,TouchTargetSizeCheck" />

    <TextView
        android:id="@+id/textView"
        android:layout width="match parent"
        android:layout height="wrap content"
        android:textSize="16sp" />

    <LinearLayout
        android:layout width="match parent"
        android:layout height="wrap content"
        android:orientation="horizontal">

        <Button
            android:id="@+id/btnDisplay"
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:layout_weight="1"
            android:text="Display" />

        <Button
            android:id="@+id/btnClear"
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:layout_weight="1"
            android:text="Clear" />
    </LinearLayout>

</LinearLayout>
```

MainActivity.kt

```
package com.example.pract7

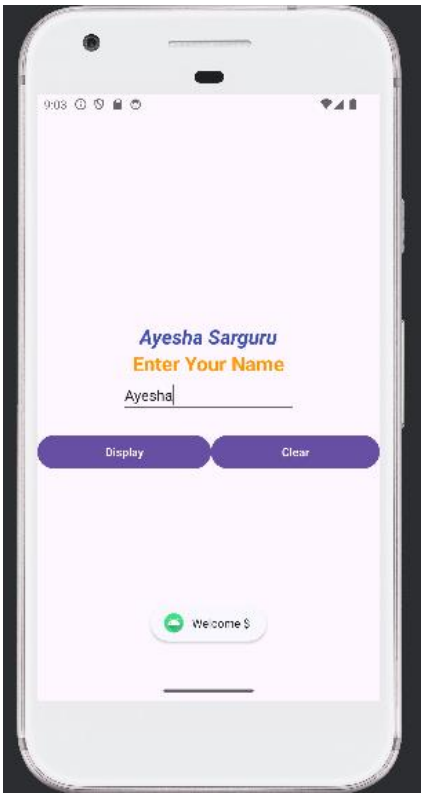
import android.os.Bundle
import android.widget.Button
import android.widget.EditText
import android.widget.Toast
import androidx.activity.enableEdgeToEdge
import androidx.appcompat.app.AppCompatActivity
import androidx.core.view.ViewCompat
import androidx.core.view.WindowInsetsCompat

class MainActivity : AppCompatActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        enableEdgeToEdge()
        setContentView(R.layout.activity_main)
        val etName=findViewById<EditText>(R.id.txtName)
        val btDisplay=findViewById<Button>(R.id.btnDisplay)
        val btClear=findViewById<Button>(R.id.btnClear)
        btDisplay.setOnClickListener {
            val name=etName.text.toString()
            if(name.isNotEmpty()){
                Toast.makeText(this,"Welcome $ ",Toast.LENGTH_SHORT).show()

            }
            else
            {
                Toast.makeText(this,"Kindly enter your Name",Toast.LENGTH_SHORT).show()
            }
        }
        btClear.setOnClickListener {
            etName.text.clear()
        }

    }
}
```

Output:



Practical 8

Aim: Develop an Android application using RelativeLayout to align UI components dynamically.

RelativeLayout positions views in relation to the parent layout or to other views. Attributes such as layout_below, layout_alignParentTop, and layout_toEndOf help control the position of UI components. It can be used to create forms and interfaces with related elements.

activity_main.xml

```
<?xml version="1.0" encoding="utf-8"?>
<RelativeLayout xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    xmlns:tools="http://schemas.android.com/tools"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:gravity="center"
    tools:context=".MainActivity">

    <TextView
        android:id="@+id/txtStudent"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Ayesha Sarguru"
        android:textColor="#3F51B5"
        android:textSize="24sp"
        android:textStyle="bold|italic" />

    <TextView
        android:id="@+id/txtView"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:layout_below="@id/txtStudent"
        android:text="Enter Your Name"
        android:textColor="#FF9800"
        android:textSize="24sp"
        android:textStyle="bold" />

    <EditText
        android:id="@+id/txtName"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:layout_below="@id/txtView"
        android:ems="10"
        android:inputType="text"
        tools:ignore="SpeakableTextPresentCheck,TouchTargetSizeCheck" />

    <Button
        android:id="@+id/btnDisplay"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:layout_below="@id/txtName"
        android:text="Display" />

    <Button
        android:id="@+id/btnClear"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:layout_below="@id/btnDisplay"
        android:text="Clear" />

</RelativeLayout>
```

MainActivity.kt

```
package com.example.pract8

import android.os.Bundle
import android.widget.Button
import android.widget.EditText
import android.widget.Toast
import androidx.activity.enableEdgeToEdge
import androidx.appcompat.app.AppCompatActivity
```

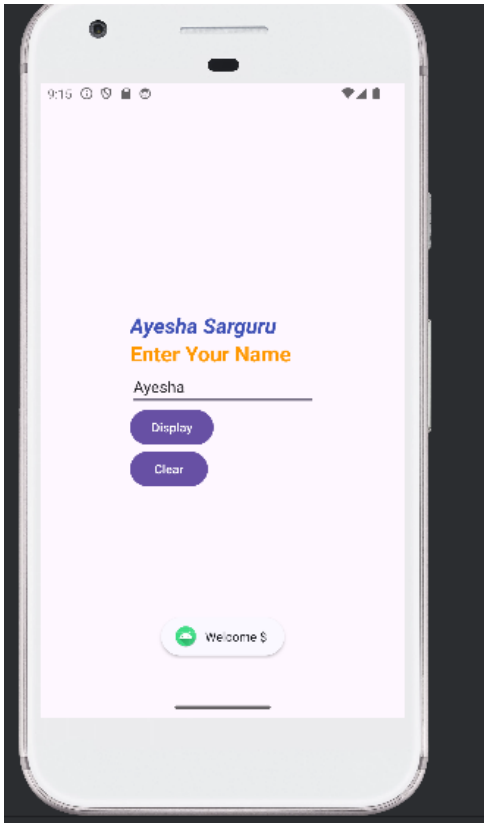
```
import androidx.core.view.ViewCompat
import androidx.core.view.WindowInsetsCompat

class MainActivity : AppCompatActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        enableEdgeToEdge()
        setContentView(R.layout.activity_main)
        val etName=findViewById<EditText>(R.id.txtName)
        val btDisplay=findViewById<Button>(R.id.btnDisplay)
        val btClear=findViewById<Button>(R.id.btnClear)
        btDisplay.setOnClickListener {
            val name=etName.text.toString()
            if(name.isNotEmpty()){
                Toast.makeText(this,"Welcome $ ",Toast.LENGTH_SHORT).show()

            }
            else
            {
                Toast.makeText(this,"Kindly enter your Name",Toast.LENGTH_SHORT).show()
            }
        }
        btClear.setOnClickListener {
            etName.text.clear()
        }

    }
}
```

Output:



Practical 9

Aim: Develop an Android application using TableLayout to design a structured Login/Registration Form.

TableLayout organises UI components into rows and columns using TableRow elements. Each row can contain labels, text fields, buttons, or other views. It is useful for designing structured login, registration, and data-entry forms.

activity_main.xml

```
<?xml version="1.0" encoding="utf-8"?>
<TableLayout xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    xmlns:tools="http://schemas.android.com/tools"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:gravity="center"
    tools:context=".MainActivity">

    <TableRow
        android:layout width="match parent"
        android:layout height="match parent">

        <TextView
            android:id="@+id/textView"
            android:layout width="wrap content"
            android:layout height="wrap content"
            android:layout_weight="1"
            android:text="User Name"
            android:textSize="24sp"
            android:textStyle="bold" />

        <EditText
            android:id="@+id/etUserName"
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:ems="10"
            android:hint="Enter User Name"
            android:inputType="text"
            android:textSize="20sp"
            tools:ignore="SpeakableTextPresentCheck" />
    </TableRow>

    <TableRow
        android:layout_width="match_parent"
        android:layout_height="match_parent">

        <TextView
            android:id="@+id/textView2"
            android:layout width="wrap content"
            android:layout_height="wrap_content"
            android:layout_weight="1"
            android:text="Password"
            android:textSize="24sp"
            android:textStyle="bold" />

        <EditText
            android:id="@+id/etPassword"
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:ems="10"
            android:hint="Enter Password"
            android:inputType="numberPassword"
            android:textSize="20sp"
            tools:ignore="SpeakableTextPresentCheck" />
    </TableRow>
    <TableRow
        android:layout_width="match_parent"
        android:layout_height="match_parent">

        <Button
            android:id="@+id/btnLogin"
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
```

```
            android:layout_weight="1"
            android:text="Login"
            android:textSize="20sp"
            android:textStyle="bold" />

        <Button
            android:id="@+id/btnCancel"
            android:layout width="wrap content"
            android:layout height="wrap content"
            android:text="Cancel"
            android:textSize="20sp"
            android:textStyle="bold" />
    </TableRow>
</TableLayout>
```

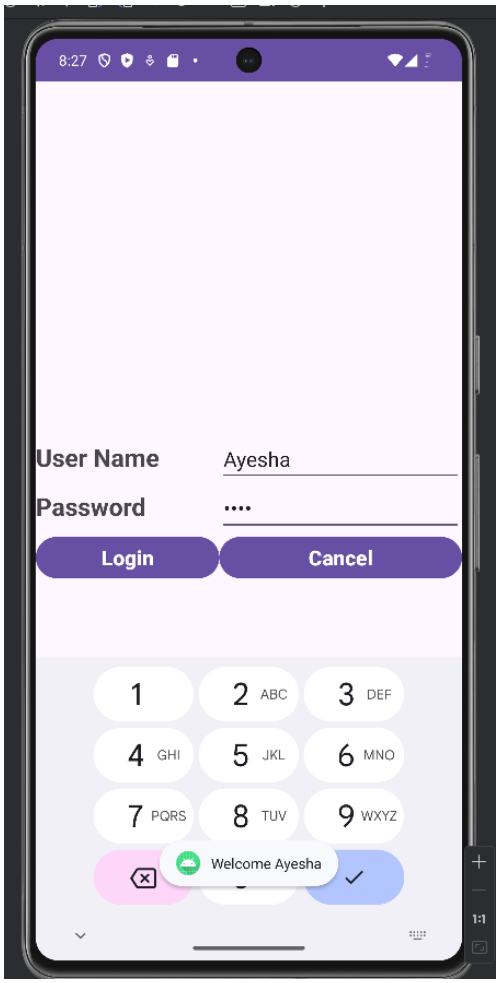
MainActivity.kt

```
package com.example.pract9

import androidx.appcompat.app.AppCompatActivity
import android.os.Bundle
import android.widget.Button
import android.widget.EditText
import android.widget.Toast

class MainActivity : AppCompatActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContentView(R.layout.activity_main)
        val etUserName=findViewById<EditText>(R.id.etUserName)
        val etPassword=findViewById<EditText>(R.id.etPassword)
        val btLogin=findViewById<Button>(R.id.btnLogin)
        val btCancel=findViewById<Button>(R.id.btnCancel)
        btLogin.setOnClickListener {
            val name=etUserName.text.toString()
            val pass=etPassword.text.toString()
            if(name=="Ayesha" && pass=="1720"){
                Toast.makeText(this,"Welcome $name",Toast.LENGTH_SHORT).show()
            }
            else
            {
                Toast.makeText(this,"KIndly Enter Correct Login details",Toast.LENGTH_SHORT).show()
            }
            btCancel.setOnClickListener {
                etUserName.text.clear()
                etPassword.text.clear()
            }
        }
    }
}
```

Output



Practical 10

Aim: Develop an Android application using ScrollView to display long scrollable content.

ScrollView allows content that is taller than the screen to be viewed by scrolling vertically. It generally contains one direct child layout, which holds the views that need scrolling. It is useful for long forms, instructions, and pages with extensive content.

activity_main.xml

```
<?xml version="1.0" encoding="utf-8"?>
<ScrollView xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    xmlns:tools="http://schemas.android.com/tools"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    tools:context=".MainActivity" >

    <LinearLayout
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:orientation="vertical">

        <ImageView
            android:id="@+id/imageView2"
            android:layout_width="match_parent"
            android:layout_height="775dp"
            app:srcCompat="@drawable/ic_launcher_foreground"
            tools:srcCompat="@drawable/ic_launcher_foreground" />

        <TextView
            android:id="@+id/textView5"
            android:layout_width="match_parent"
            android:layout_height="wrap_content"
            android:text="Andriod Programming"
            android:textSize="40sp"
            android:textStyle="bold" />

        <TextView
            android:id="@+id/textView6"
            android:layout_width="match_parent"
            android:layout_height="wrap_content"
            android:text="Unit1:Introduction to Andriod"
            android:textSize="40sp"
            android:textStyle="italic" />

        <TextView
            android:id="@+id/textView7"
            android:layout_width="match_parent"
            android:layout_height="wrap_content"
            android:text="Unit 2:Actvities and intents"
            android:textSize="40sp" />

        <TextView
            android:id="@+id/textView8"
            android:layout_width="match_parent"
            android:layout_height="wrap_content"
            android:text="Unit 3:Layouts"
            android:textSize="40sp" />

        <TextView
            android:id="@+id/textView9"
            android:layout_width="match_parent"
            android:layout_height="wrap_content"
            android:text="Unit 4:Fragments"
            android:textSize="40sp" />

        <TextView
            android:id="@+id/textView10"
            android:layout_width="match_parent"
            android:layout_height="wrap_content"
            android:text="Unit 5:SQLite Database"
            android:textSize="40sp" />

        <TextView
            android:id="@+id/textView11"
```

```
            android:layout_width="match_parent"
            android:layout_height="wrap_content"
            android:text="unit6: Networking"
            android:textSize="40sp" />

        <TextView
            android:id="@+id/textView12"
            android:layout_width="match_parent"
            android:layout_height="wrap_content"
            android:text="Unit 7:Firebase"
            android:textSize="40sp" />

        <TextView
            android:id="@+id/textView13"
            android:layout_width="match_parent"
            android:layout_height="wrap_content"
            android:text="Unit 8:Publishing Apps"
            android:textSize="40sp" />

    </LinearLayout>
</ScrollView>
```

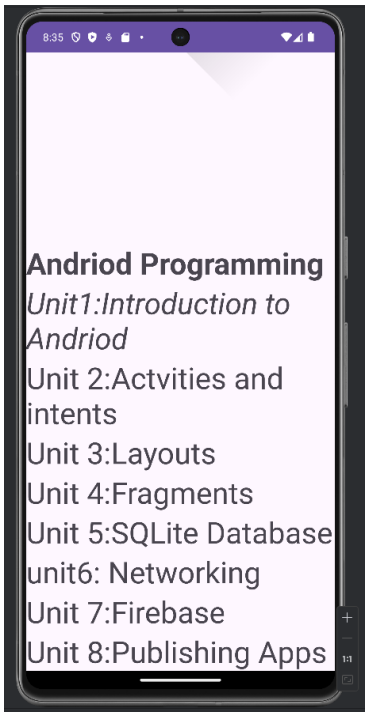
MainActivity.kt

```
package com.example.pract10

import androidx.appcompat.app.AppCompatActivity
import android.os.Bundle

class MainActivity : AppCompatActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContentView(R.layout.activity_main)
    }
}
```

Output



Practical 11

Aim: Develop an Android application using AbsoluteLayout demonstrating fixed positioning.

AbsoluteLayout places views at fixed x and y coordinates. This gives direct control over the position of each component, but the interface may not adjust well to different screen sizes or resolutions. It is a legacy layout and is generally avoided in modern responsive Android design.

activity_main.xml

```
<?xml version="1.0" encoding="utf-8"?>
<AbsoluteLayout xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    xmlns:tools="http://schemas.android.com/tools"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    tools:context=".MainActivity">

    <TextView
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:layout_x="20dp"
        android:layout_y="40dp"
        android:text="UserName"
        android:textSize="20sp" />

    <EditText
        android:id="@+id/etUserName"
        android:layout_width="200dp"
        android:layout_height="wrap_content"
        android:layout_x="120dp"
        android:layout_y="35dp"
        android:ems="10"
        android:hint="Enter UserName"
        android:inputType="text"
        tools:ignore="TouchTargetSizeCheck" />

    <Button
        android:id="@+id/btnLogin"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:layout_x="120dp"
        android:layout_y="100dp"
        android:text="Login" />
</AbsoluteLayout>
```

MainActivity.kt

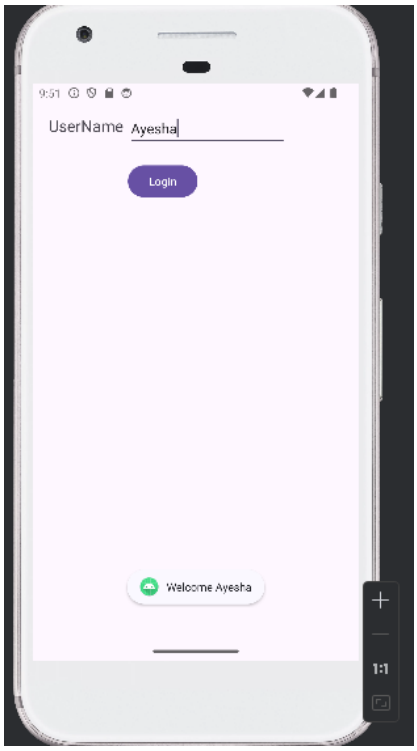
```
package com.example.pract11

import android.os.Bundle
import android.widget.TextView
import android.widget.Toast
import androidx.activity.enableEdgeToEdge
import androidx.appcompat.app.AppCompatActivity
import androidx.core.view.ViewCompat
import androidx.core.view.WindowInsetsCompat

class MainActivity : AppCompatActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        enableEdgeToEdge()
        setContentView(R.layout.activity_main)
        val etUsername=findViewById<TextView>(R.id.etUserName)
        val btLogin=findViewById<TextView>(R.id.btnLogin)
        btLogin.setOnClickListener {
            val name = etUsername.text.toString()
            Toast.makeText(this,"Welcome $name", Toast.LENGTH_SHORT).show()
        }

    }
}
```

Output



Practical 12

Aim: Develop an Android application using FrameLayout to overlay UI elements.

FrameLayout is a simple container that can display child views in a stack. Views can overlap, making it suitable for overlays, fragment containers, and displaying content on top of images. The order of child views affects which view appears in front.

activity_main.xml

```
<?xml version="1.0" encoding="utf-8"?>
<FrameLayout xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    xmlns:tools="http://schemas.android.com/tools"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    tools:context=".MainActivity" >

    <ImageView
        android:id="@+id/imageView"
        android:layout_width="match_parent"
        android:layout_height="481dp"
        android:scaleType="center"

        app:srcCompat="@drawable/img" />

    <TextView
        android:id="@+id/ImageView"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:layout_gravity="center"
        android:text="Ayesha"
        android:textColor="@color/white"
        android:textSize="48sp" />

</FrameLayout>
```

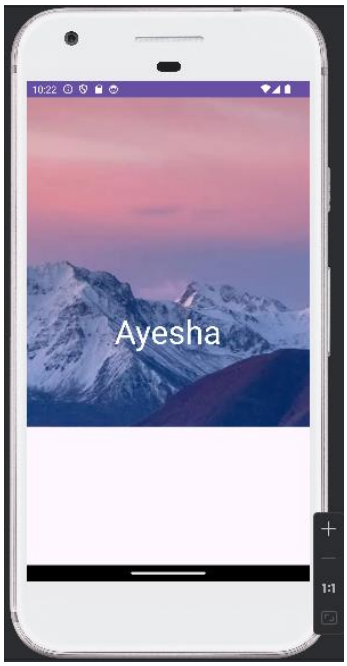
MainActivity.kt

```
package com.example.pract12

import androidx.appcompat.app.AppCompatActivity
import android.os.Bundle

class MainActivity : AppCompatActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContentView(R.layout.activity_main)
    }
}
```

Output



Practical 13

Aim: Develop an Android application using ListView with ArrayAdapter and implement item click handling.

ListView displays a vertically scrollable list of items. ArrayAdapter connects an array or list of data to the ListView and provides a view for each item. An item-click listener can be used to respond when the user selects an item.

activity_main.xml

```
<?xml version="1.0" encoding="utf-8"?>
<ListView xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    xmlns:tools="http://schemas.android.com/tools"
    android:id="@+id/listView"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:entries="@array/list"
    tools:context=".MainActivity" />
```

MainActivity.kt

```
package com.example.pract13

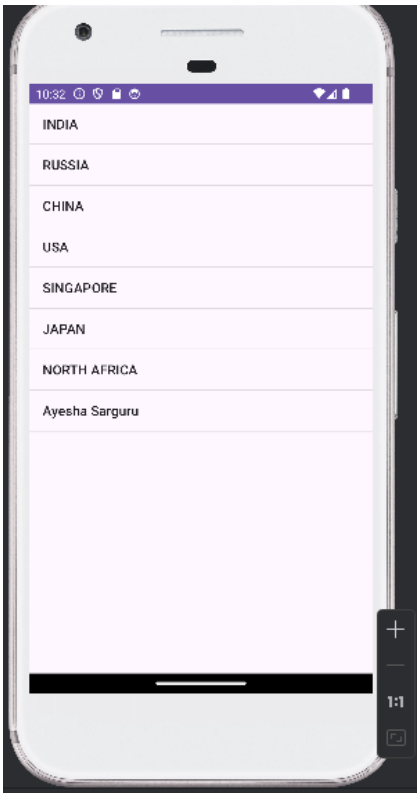
import androidx.appcompat.app.AppCompatActivity
import android.os.Bundle
import android.widget.ArrayAdapter
import android.widget.ListView
import android.widget.Toast

class MainActivity : AppCompatActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContentView(R.layout.activity_main)
        val listView =findViewById<ListView>(R.id.listView)
        val items=resources.getStringArray(R.array.list)
        val adapter =ArrayAdapter(this,android.R.layout.simple list item 1, items)
        listView.adapter=adapter
        listView.setOnItemClickListener { _,_, position, _ ->
            Toast.makeText(this, items[position],Toast.LENGTH_SHORT).show()
        }
    }
}
```

strings.xml

```
<resources>
    <string name="app_name">pract13</string>
    <array name="list">
        <item>INDIA</item>
        <item>RUSSIA</item>
        <item>CHINA</item>
        <item>USA</item>
        <item>SINGAPORE</item>
        <item>JAPAN</item>
        <item>NORTH AFRICA</item>
        <item>Ayesha Sarguru</item>
    </array>
</resources>
```

Output


10:32
INDIA
RUSSIA
CHINA
USA
SINGAPORE
JAPAN
NORTH AFRICA
Ayesha Sarguru
1:1

Practical 14

Aim: Develop an Android application using GridView with Custom Adapter and image selection handling.

GridView displays items in a two-dimensional, scrollable grid of rows and columns. A custom adapter can define how each item, such as an image and a label, is displayed. An item-click listener can handle image or grid-item selection.

activity_main.xml

```
<?xml version="1.0" encoding="utf-8"?>
<GridView xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    xmlns:tools="http://schemas.android.com/tools"
    android:id="@+id/GridView"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:visibility="visible"
    tools:context=".MainActivity">


</GridView>
```

MainActivity.kt

```
package com.example.pract14

import MyAdapter
import android.annotation.SuppressLint
import androidx.appcompat.app.AppCompatActivity
import android.os.Bundle
import android.widget.GridView
import android.widget.Toast
class MainActivity : AppCompatActivity() {
            private val names = arrayOf(
                "apple",
                "banana",
                "mango",
                "orange"
            )
            private val images =
                intArrayOf(R.drawable.apple, R.drawable.banana, R.drawable.mango, R.drawable.orange)

            @SuppressLint("MissingInflatedId")
            override fun onCreate(savedInstanceState: Bundle?) {
                super.onCreate(savedInstanceState)
                setContentView(R.layout.activity_main)
                val gridView = findViewById<GridView>(R.id.GridView)
                val adapter = MyAdapter(this, names, images)
                gridView.adapter = adapter
                gridView.setOnItemClickListener { _, _, position, _ ->
                    Toast.makeText(this, names[position], Toast.LENGTH_LONG).show()

                }
            }
        }
```

grid_item.xml

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    xmlns:tools="http://schemas.android.com/tools"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:gravity="center"
    android:orientation="vertical"
    tools:context=".MyAdapter">

    <ImageView
        android:id="@+id/imageView3"
        android:layout width="80dp"
        android:layout height="80dp"
        app:srcCompat="@mipmap/ic_launcher" />

    <TextView
```

```
        android:id="@+id/textView"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:gravity="center"
        android:text="Apple"
        android:textColor="#C51111"
        android:textSize="24sp"
        android:textStyle="bold" />
</LinearLayout>
```

MyAdapter.kt

```
import android.content.Context
import android.view.LayoutInflater
import android.view.View
import android.view.ViewGroup
import android.widget.BaseAdapter
import android.widget.ImageView
import android.widget.TextView
import com.example.pract14.R

class MyAdapter(
    private val context: Context,
    private val names: Array<String>,
    private val images: IntArray
) : BaseAdapter() {

    override fun getCount(): Int = names.size

    override fun getItem(position: Int): Any = names[position]

    override fun getItemId(position: Int): Long = position.toLong()

    override fun getView(position: Int, convertView: View?, parent: ViewGroup?): View {

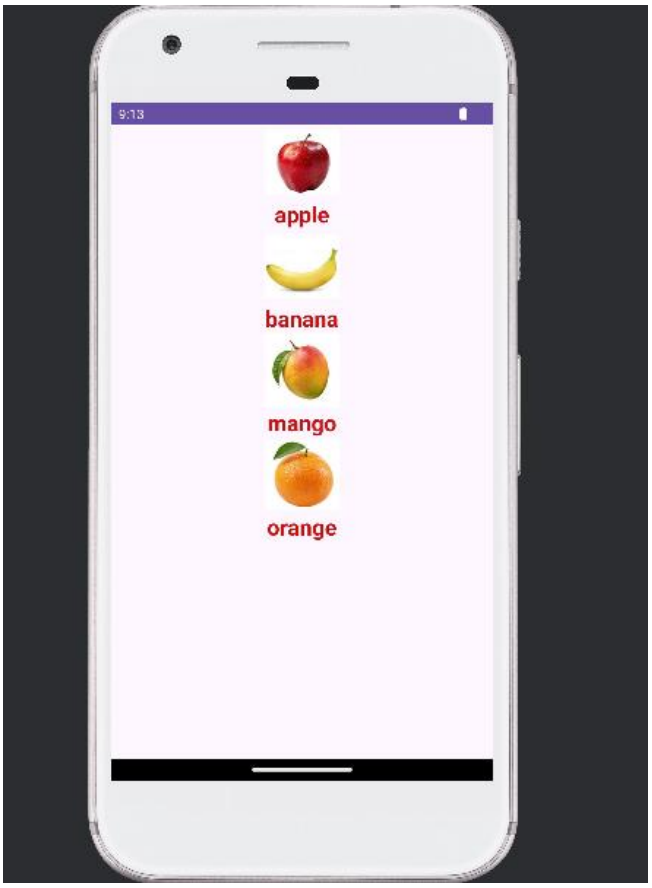
        val view = LayoutInflater.from(context).inflate(R.layout.grid item, parent, false)

        val image = view.findViewById<ImageView>(R.id.imageView3)
        val text = view.findViewById<TextView>(R.id.textView)

        image.setImageResource(images[position])
        text.text = names[position]

        return view
    }
}
```

Output



Practical 15

Aim: Develop an Android application implementing RecyclerView as a modern alternative to ListView.

RecyclerView is a flexible and efficient component for displaying large sets of data. It uses a ViewHolder to reuse item views and an Adapter to bind data to those views. A LayoutManager controls whether items appear in a list, grid, or another arrangement.

activity_main.xml

```
<?xml version="1.0" encoding="utf-8"?>
<androidx.recyclerview.widget.RecyclerView
    xmlns:android="http://schemas.android.com/apk/res/android"
    android:id="@+id/recyclerView"
    android:layout_width="match_parent"
    android:layout_height="match_parent"/>
```

MainActivity.kt

```
package com.example.pract15

import android.annotation.SuppressLint
import androidx.appcompat.app.AppCompatActivity
import android.os.Bundle
import android.widget.GridView
import android.widget.Toast
import androidx.recyclerview.widget.LinearLayoutManager
import androidx.recyclerview.widget.RecyclerView

class MainActivity : AppCompatActivity() {

    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContentView(R.layout.activity_main)

        val recyclerView = findViewById<RecyclerView>(R.id.recyclerView)

        val items = arrayOf(
            "Apple",
            "Banana",
            "Mango",
            "Orange",

        )

        recyclerView.layoutManager = LinearLayoutManager(this)
        recyclerView.adapter = MyAdapter(items)
    }
}
```

item.xml

```
<?xml version="1.0" encoding="utf-8"?>
<TextView xmlns:android="http://schemas.android.com/apk/res/android"
    android:id="@+id/textView"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:padding="16dp"
    android:text="Item"
    android:textSize="18sp"/>
```

MyAdapter.kt

```
package com.example.pract15

import android.view.LayoutInflater
import android.view.View
import android.view.ViewGroup
import android.widget.TextView
import androidx.recyclerview.widget.RecyclerView

class MyAdapter(private val items: Array<String>) :
```

```
    RecyclerView.Adapter<MyAdapter.ViewHolder>() {

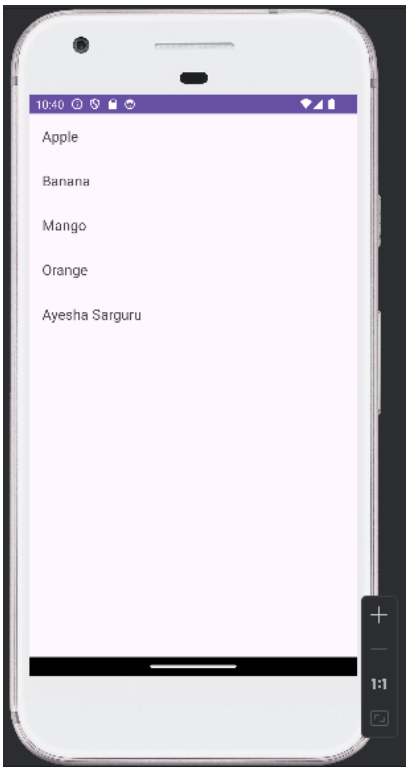
    class ViewHolder(view: View) : RecyclerView.ViewHolder(view) {
        val text = view.findViewById<TextView>(R.id.textView)
    }

    override fun onCreateViewHolder(parent: ViewGroup, viewType: Int): ViewHolder {
        val view = LayoutInflater.from(parent.context)
            .inflate(R.layout.item, parent, false)
        return ViewHolder(view)
    }

    override fun onBindViewHolder(holder: ViewHolder, position: Int) {
        holder.text.text = items[position]
    }

    override fun getItemCount(): Int = items.size
}
```

Output



MODULE 2

Practical 1

Aim: Develop an Android application implementing AppBar (Toolbar) and Options Menu.

The Toolbar provides a consistent top area for an application's title, navigation controls, and actions. An options menu presents additional actions that users can select. Menu resources and menu callbacks are used to define and handle these actions.

activity_main.xml

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    xmlns:tools="http://schemas.android.com/tools"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:orientation="vertical"
    tools:context=".MainActivity">

    <androidx.appcompat.widget.Toolbar
        android:id="@+id/toolbar"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:background="?attr/colorPrimary"
        android:minHeight="?attr/actionBarSize"
        android:theme="?attr/actionBarTheme"
        app:title="Ayesha Sarguru" />

    <TextView
        android:layout_width="match_parent"
        android:layout_height="match_parent"
        android:gravity="center"
        android:text="Welcome"
        android:textSize="34sp"
        android:textStyle="bold" />

</LinearLayout>
```

Menu.xml

```
<?xml version="1.0" encoding="utf-8"?>
<menu xmlns:android="http://schemas.android.com/apk/res/android">

    <item
        android:id="@+id/help"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:title="Help" />
    <item
        android:id="@+id/view"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:title="View">
        <menu>
            <item
                android:id="@+id/logout"
                android:title="Logout" />
        </menu>
    </item>

</menu>
```

MainActivity.kt

```
package com.example.m2pract1

import androidx.appcompat.app.AppCompatActivity
import android.os.Bundle
import android.view.*
import android.view.Menu
import android.view.MenuItem
```

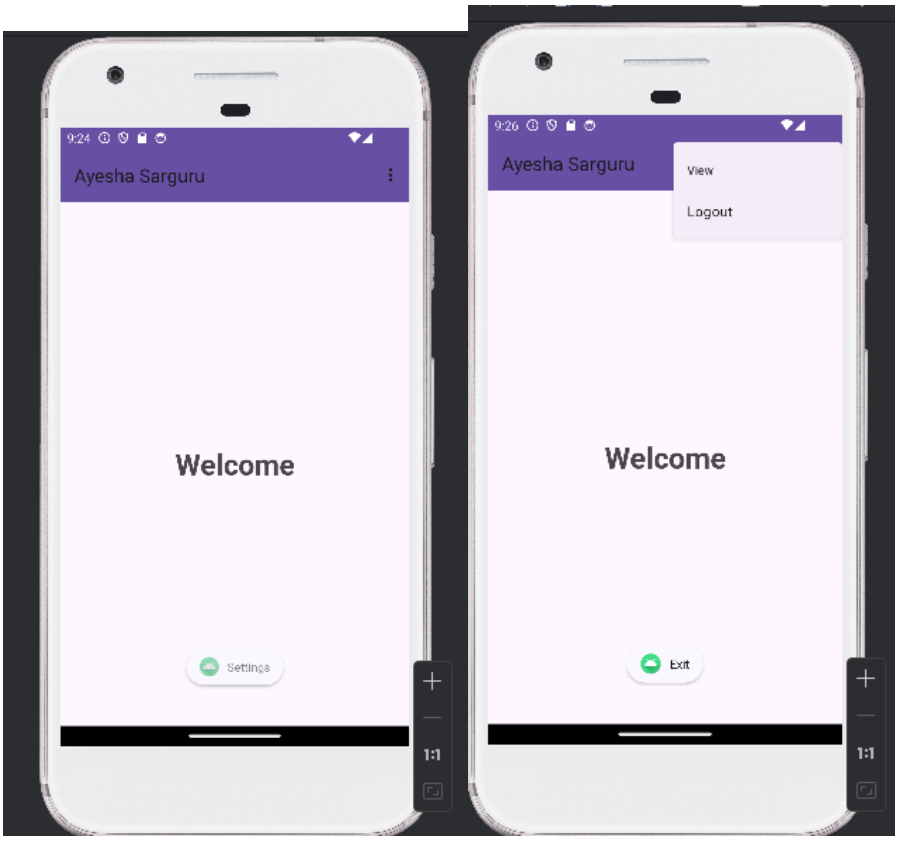
```
import android.widget.Toast
import androidx.appcompat.widget.Toolbar

class MainActivity : AppCompatActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContentView(R.layout.activity_main)

        val toolbar = findViewById<Toolbar>(R.id.toolbar)
        setSupportActionBar(toolbar)
    }
        override fun onCreateOptionsMenu(menu: Menu): Boolean{
            menuInflater.inflate(R.menu.menu,menu)
            return true
        }

        override fun onOptionsItemSelected(item: MenuItem): Boolean {
            when (item.itemId){
                R.id.help ->{
                    Toast.makeText(this,"Settings",Toast.LENGTH_SHORT).show()
                    return true
                }
                R.id.view ->{
                    Toast.makeText(this, "Exit", Toast.LENGTH_SHORT).show()
                    return true
                }
                R.id.logout ->{
                    Toast.makeText(this, "Logged Out", Toast.LENGTH_SHORT).show()
                    return true
                }
            }
            return super.onOptionsItemSelected(item)
        }
    }
```

Output



Practical 2

Aim: Develop an Android application demonstrating AlertDialog and custom DialogFragment.

An AlertDialog displays a small dialog containing a title, message, and action buttons, such as OK or Cancel. A custom DialogFragment can display a dialog with a customised layout. Dialogs are useful for confirmations, warnings, and short user interactions.

activity_main.xml

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    xmlns:tools="http://schemas.android.com/tools"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:gravity="center"
    android:orientation="vertical"
    tools:context=".MainActivity">

    <Button
        android:id="@+id/btnAlert"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Alert Dialog" />

    <Button
        android:id="@+id/btnCustom"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Custom Dialog" />

</LinearLayout>
```

dialog_custom.xml

```
<?xml version="1.0" encoding="utf-8"?>

<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    xmlns:tools="http://schemas.android.com/tools"
    android:id="@+id/main"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:orientation="vertical"
    android:padding="20dp"
    tools:context=".CustomDialogFragment">

    <EditText
        android:id="@+id/etName"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:ems="10"
        android:hint="Enter Name"
        android:inputType="text"
        android:textSize="24sp"
        tools:ignore="TouchTargetSizeCheck" />

</LinearLayout>
```

CustomDialogFragment.kt

```
package com.example.m2pract2

import android.app.Dialog
import android.os.Bundle
import android.view.LayoutInflater
import androidx.appcompat.app.AlertDialog
import androidx.fragment.app.DialogFragment

class CustomDialogFragment : DialogFragment() {

    override fun onCreateDialog(savedInstanceState: Bundle?): Dialog {
```

```
        val view = LayoutInflater.from(requireContext())
            .inflate(R.layout.dialog_custom, null)

        return AlertDialog.Builder(requireContext())
            .setTitle("Custom Dialog")
            .setView(view)
            .setPositiveButton("OK", null)
            .show()
    }
}
```

MainActivity.kt

```
package com.example.m2pract2

import android.os.Bundle
import android.widget.Button
import androidx.appcompat.app.AlertDialog
import androidx.appcompat.app.AppCompatActivity

class MainActivity : AppCompatActivity() {

    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)

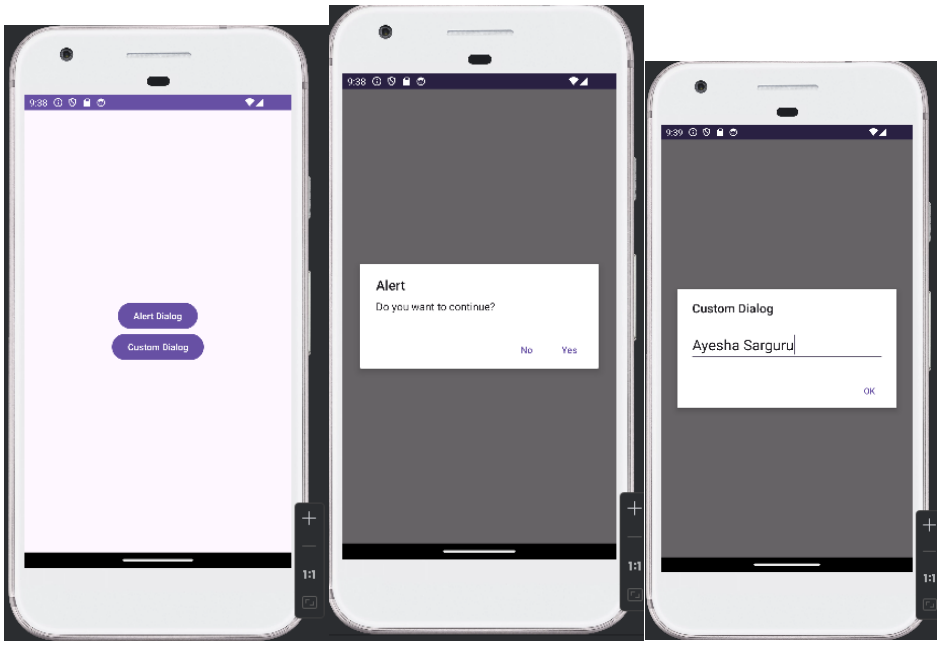
        setContentView(R.layout.activity_main)

        val btnAlert = findViewById<Button>(R.id.btnAlert)
        val btnCustom = findViewById<Button>(R.id.btnCustom)

        btnAlert.setOnClickListener {
            AlertDialog.Builder(this)
                .setTitle("Alert")
                .setMessage("Do you want to continue?")
                .setPositiveButton("Yes", null)
                .setNegativeButton("No", null)
                .show()
        }

        btnCustom.setOnClickListener {
            CustomDialogFragment().show(supportFragmentManager, "CustomDialog")
        }
    }
}
```

Output:



Practical 3

Aim: Develop an Android application implementing contextual menus and popup menus.

A contextual menu provides actions related to a selected view or item, while a PopupMenu displays a compact menu anchored to a particular view. Menu items can be defined in XML or created in code. Listener methods handle the action selected by the user.

activity_main.xml

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    xmlns:tools="http://schemas.android.com/tools"
    android:layout width="match parent"
    android:layout_height="match_parent"
    android:gravity="center"
    android:orientation="vertical"
    tools:context=".MainActivity">

    <TextView
        android:id="@+id/textView2"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Ayesha Sarguru"
        android:textColor="#2196F3"
        android:textSize="24sp"
        android:textStyle="bold|italic" />

    <TextView
        android:id="@+id/textView"
        android:layout width="wrap content"
        android:layout_height="wrap_content"
        android:padding="20dp"
        android:text="Long Press Me"
        android:textColor="#FF9800"
        android:textSize="24sp"
        android:textStyle="bold" />

    <Button
        android:id="@+id/button"
        android:layout width="wrap content"
        android:layout height="wrap content"
        android:text="Popup Menu"
        android:textSize="20sp"
        android:textStyle="bold" />

</LinearLayout>
```

popup_menu.xml

```
<?xml version="1.0" encoding="utf-8"?>
<menu xmlns:android="http://schemas.android.com/apk/res/android">

    <item
        android:id="@+id/menuShare"
        android:layout width="wrap content"
        android:layout height="wrap content"
        android:title="Share" />
    <item
        android:id="@+id/menuSettings"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:title="Settings" />
</menu>
```

MainActivity.kt

```
package com.example.m2pract3

import android.os.Bundle
import android.view.ContextMenu
import android.view.MenuItem
import android.view.View
```

```
import android.widget.Button
import android.widget.PopupMenu
import android.widget.TextView
import android.widget.Toast
import androidx.appcompat.app.AppCompatActivity

class MainActivity : AppCompatActivity() {

    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContentView(R.layout.activity_main)

        val textView = findViewById<TextView>(R.id.textView)
        val button = findViewById<Button>(R.id.button)

        registerForContextMenu(textView)

        button.setOnClickListener {
            val popup = PopupMenu(this, button)

            popup.menuInflater.inflate(R.menu.popup_menu, popup.menu)

            popup.setOnMenuItemClickListener {
                Toast.makeText(
                    this,
                    it.title,
                    Toast.LENGTH_SHORT
                ).show()
                true
            }

            popup.show()
        }
    }

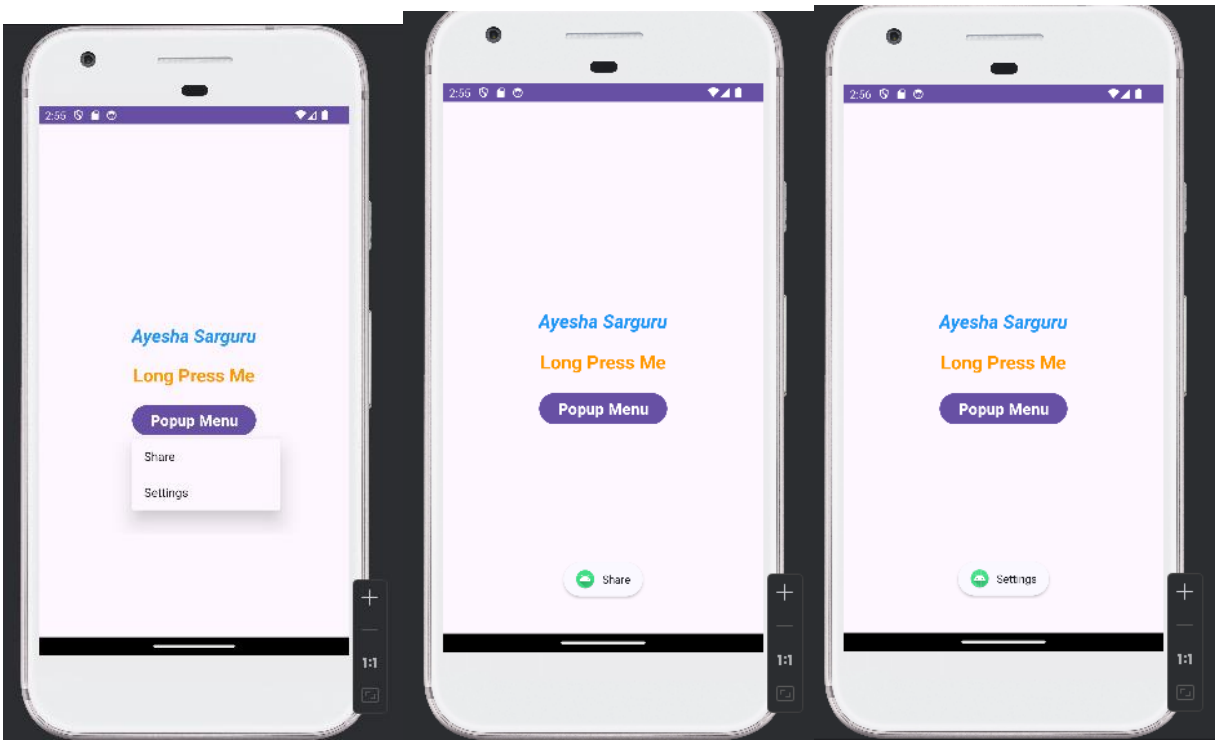
    override fun onCreateContextMenu(
        menu: ContextMenu,
        v: View,
        menuInfo: ContextMenu.ContextMenuInfo?
    ) {
        super.onCreateContextMenu(menu, v, menuInfo)

        menu.setHeaderTitle("Select Action")
        menu.add("Edit")
        menu.add("Delete")
    }

    override fun onContextItemSelected(item: MenuItem): Boolean {
        Toast.makeText(
            this,
            item.title,
            Toast.LENGTH_SHORT
        ).show()

        return true
    }
}
```

Output:


Ayesha Sarguru
Long Press Me
Popup Menu
Share
Settings
Ayesha Sarguru
Long Press Me
Popup Menu
Share
Ayesha Sarguru
Long Press Me
Popup Menu
Settings

Practical 4

Aim: Develop an Android application implementing Explicit and Implicit Intents.

An Intent is a messaging object used to request an action from another Android component. An explicit Intent names the component to open, such as a particular Activity. An implicit Intent describes an action, such as opening a webpage or sharing text, and Android finds a suitable application to handle it.

activity_main.xml

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    xmlns:tools="http://schemas.android.com/tools"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:gravity="center"
    android:orientation="vertical"
    tools:context=".MainActivity">

    <Button
        android:id="@+id/btnExplicit"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Explicit Intent"
        android:textSize="24sp"
        android:textStyle="bold" />

    <Button
        android:id="@+id/btnImplicit"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Implicit Intent"
        android:textSize="24sp"
        android:textStyle="bold" />
</LinearLayout>
```

activity_second.xml

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    xmlns:tools="http://schemas.android.com/tools"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:gravity="center"
    android:orientation="vertical"
    tools:context=".SecondActivity">

    <TextView
        android:id="@+id/textView"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Second Activity"
        android:textColor="#FF9800"
        android:textSize="34sp"
        android:textStyle="bold" />
</LinearLayout>
```

Create Second Activity

MainActivity.kt

```
package com.example.m2pract4

import android.content.Intent
import android.net.Uri
import android.os.Bundle
import android.widget.Button
import androidx.appcompat.app.AppCompatActivity

class MainActivity : AppCompatActivity() {
```

```
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContentView(R.layout.activity_main)

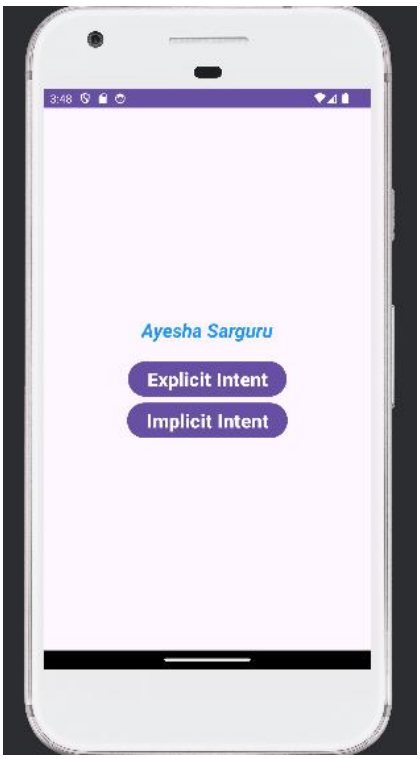
        val btnExplicit = findViewById<Button>(R.id.btnExplicit)
        val btnImplicit = findViewById<Button>(R.id.btnImplicit)

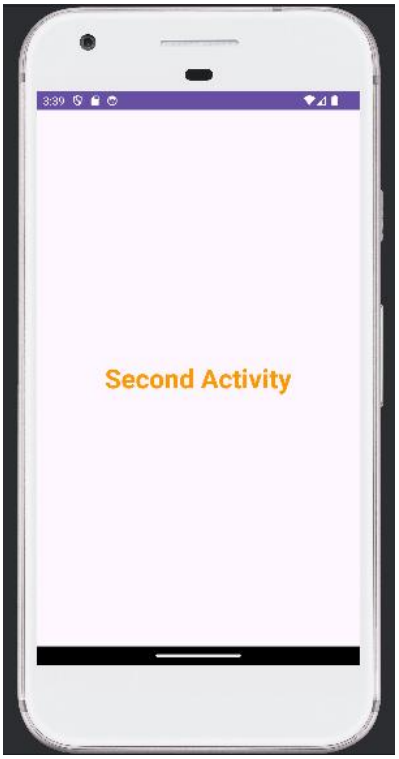
        btnExplicit.setOnClickListener {
            val intent = Intent(this, SecondActivity::class.java)
            startActivity(intent)
        }

        btnImplicit.setOnClickListener {
            val intent = Intent(
                Intent.ACTION_VIEW,
                Uri.parse("https://www.google.com")
            )
            startActivity(intent)
        }
    }
}
```

Output:

Explicit **Implicit**







Practical 5

Aim: Develop an Android application demonstrating Event Handling using Listeners (OnClick, OnLongClick, etc.).

Event handling allows an application to respond to user actions such as clicks, long presses, and selections. Listener interfaces, including OnClickListener and OnLongClickListener, are attached to views to detect these actions. The corresponding callback contains the code that should run when an event occurs.

activity_main.xml

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    xmlns:tools="http://schemas.android.com/tools"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:gravity="center"
    android:orientation="vertical"
    tools:context=".MainActivity">

    <TextView
        android:id="@+id/textView"
        android:layout_width="wrap_content"
        android:layout height="wrap content"
        android:text="Ayesha Sarguru"
        android:textColor="#2196F3"
        android:textSize="34sp"
        android:textStyle="bold|italic" />

    <Button
        android:id="@+id/button"
        android:layout width="wrap content"
        android:layout height="wrap content"
        android:text="Click Me"
        android:textSize="24sp"
        android:textStyle="bold" />

</LinearLayout>
```

MainActivity.kt

```
package com.example.m2pract5

import android.os.Bundle
import android.widget.Button
import android.widget.Toast
import androidx.appcompat.app.AppCompatActivity

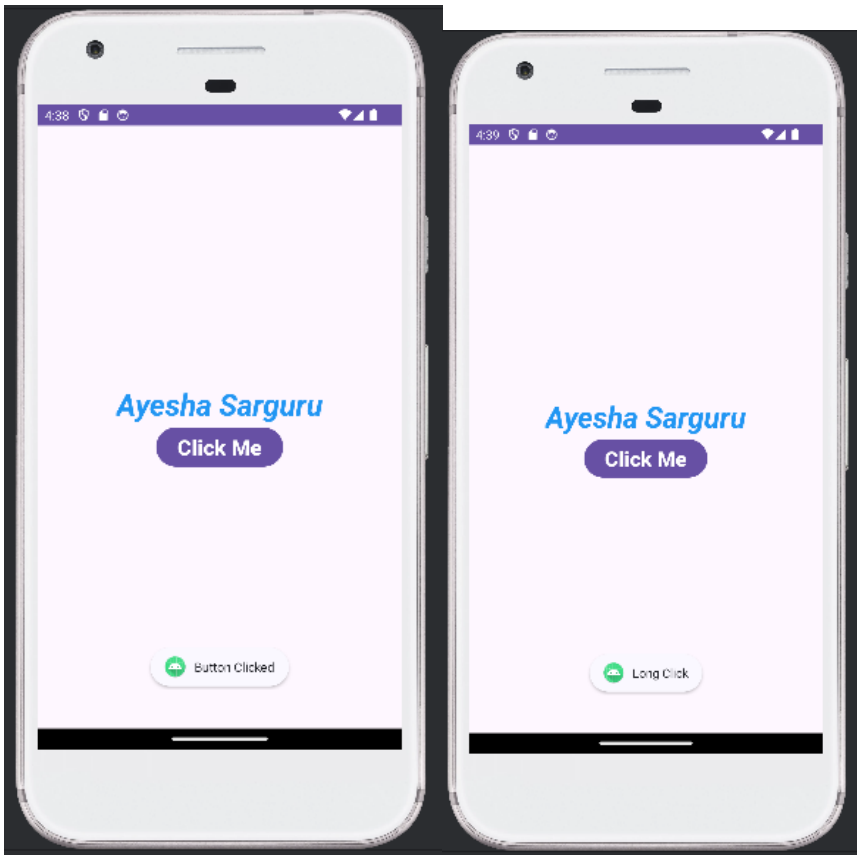
class MainActivity : AppCompatActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)sas
        setContentView(R.layout.activity main)

        val button = findViewById<Button>(R.id.button)

        button.setOnClickListener {
            Toast.makeText(this, "Button Clicked", Toast.LENGTH_SHORT).show()
        }

        button.setOnLongClickListener {
            Toast.makeText(this, "Long Click", Toast.LENGTH_SHORT).show()
            true
        }
    }
}
```

Output:


4:38
Ayesha Sarguru
Click Me
Button Clicked
4:39
Ayesha Sarguru
Click Me
Long Click

Practical 6

Aim: Develop an Android application demonstrating Adapters to populate Spinner or ListView dynamically.

An Adapter connects a data source to a UI component and provides views for displaying individual data items. A Spinner presents a drop-down selection list, while a ListView presents a scrollable list. Adapters can populate these components dynamically when data changes.

activity_main.xml

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    xmlns:tools="http://schemas.android.com/tools"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:gravity="center"
    android:orientation="vertical"
    tools:context=".MainActivity">

    <Spinner
        android:id="@+id/spinner"
        android:layout_width="200dp"
        android:layout_height="wrap_content"
        tools:ignore="TouchTargetSizeCheck" />

    <TextView

        android:id="@+id/textView"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:gravity="center"
        android:text="Ayesha"
        android:textColor="#25327A"
        android:textSize="34sp"
        android:textStyle="bold" />

</LinearLayout>
```

Arrays.xml

```
<?xml version="1.0" encoding="utf-8"?>
<resources>
    <string-array name="list">
        <item>Jalan</item>
        <item>MET</item>
        <item>DALMIA</item>
    </string-array>
</resources>
```

MainActivity.kt

```
package com.example.pract62

import android.os.Bundle
import android.widget.ArrayAdapter
import android.widget.Spinner
import android.widget.Toast
import androidx.activity.enableEdgeToEdge
import androidx.appcompat.app.AppCompatActivity
import androidx.core.view.ViewCompat
import androidx.core.view.WindowInsetsCompat

class MainActivity : AppCompatActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        enableEdgeToEdge()
        setContentView(R.layout.activity_main)


        val spinner = findViewById<Spinner>(R.id.spinner)

        val items = resources.getStringArray(R.array.list)

        val adapter = ArrayAdapter(
```

```
            this,
            android.R.layout.simple_spinner_item,
            items
        )

        adapter.setDropDownViewResource(
            android.R.layout.simple_spinner_dropdown_item
        )

        spinner.adapter = adapter

        spinner.setOnItemSelectedListener(
            object : android.widget.AdapterView.OnItemSelectedListener {

                override fun onItemSelected(
                    parent: android.widget.AdapterView<*>?,
                    view: android.view.View?,
                    position: Int,
                    id: Long
                ) {
                    Toast.makeText(this@MainActivity,items[position], Toast.LENGTH_SHORT).show()
                }

                override fun onNothingSelected(
                    parent: android.widget.AdapterView<*>?
                ) {
                    // Nothing selected
                }
            }
        )
    }
}
```

Strings.xml

```
<?xml version="1.0" encoding="utf-8"?>
<resources>
    <string name="app_name">Pract62</string>
</resources>
```

Output:



Practical 7

Aim: Develop an Android application implementing Started and Bound Services.

A Service is an Android component used to perform work without directly providing a user interface. A started service is started to perform an operation, while a bound service allows other components to interact with it through a connection. Services must be managed according to Android's lifecycle and background-execution rules.

actvity_main.xml

```
<?xml version="1.0" encoding="utf-8"?>


<LinearLayout
    xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:gravity="center"
    android:orientation="vertical">

    <Button
        android:id="@+id/btnStart"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Start Service" />

    <Button
        android:id="@+id/btnStop"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Stop Service" />

    <Button
        android:id="@+id/btnBind"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Bind Service" />

</LinearLayout>
```

mainActivty.kt

```
package com.example.pract27



import android.content.Intent
import android.os.Bundle
import android.widget.Button
import androidx.appcompat.app.AppCompatActivity

class MainActivity : AppCompatActivity() {

    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContentView(R.layout.activity_main)

        val service = Intent(this, MyService::class.java)

        findViewById<Button>(R.id.btnStart).setOnClickListener {
            startService(service)
        }

        findViewById<Button>(R.id.btnStop).setOnClickListener {
            stopService(service)
        }

        findViewById<Button>(R.id.btnBind).setOnClickListener {
            bindService(
                service,
                object : android.content.ServiceConnection {
                    override fun onServiceConnected(
                        name: android.content.ComponentName?,
```

```
                        service: android.os.IBinder?
                    ) {
                    }

                    override fun onServiceDisconnected(
                        name: android.content.ComponentName?
                    ) {
                    }
                },
                BIND_AUTO_CREATE
            )
        }
    }
}
```

My service.kt

```
package com.example.pract27



package com.example.practical22

import android.app.Service
import android.content.Intent
import android.os.Binder
import android.os.IBinder
import android.widget.Toast

class MyService : Service() {

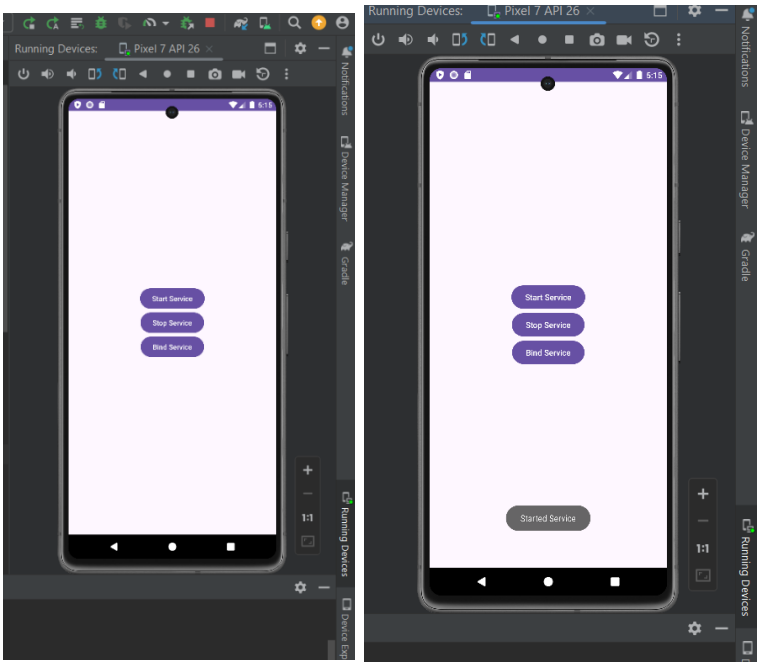

    private val binder = Binder()

    override fun onStartCommand(
        intent: Intent?,
        flags: Int,
        startId: Int
    ): Int {
        Toast.makeText(this, "Started Service", Toast.LENGTH_SHORT).show()
        return START_STICKY
    }

    override fun onBind(intent: Intent?): IBinder {
        Toast.makeText(this, "Bound Service", Toast.LENGTH_SHORT).show()
        return binder
    }

    override fun onDestroy() {
        Toast.makeText(this, "Service Stopped", Toast.LENGTH_SHORT).show()
        super.onDestroy()
    }
}
```

Output


Running Devices:
Pixel 7 API 26
Start Service
Stop Service
Bind Service
Started Service

Practical 8

Aim: Develop an Android application implementing Notifications using Notification Manager.

A notification displays information or alerts to the user outside the application's main screen. NotificationManager is used to issue and manage notifications, while a notification channel is required for most notifications on Android 8.0 and later. The application may need notification permission on newer Android versions.

activity_main.xml

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    xmlns:tools="http://schemas.android.com/tools"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:gravity="center"
    android:orientation="vertical"
    tools:context=".MainActivity">

    <TextView
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Ayesha Sarguru"
        android:textColor="#03A9F4"
        android:textSize="24sp"
        android:textStyle="bold|italic" />

    <Button
        android:id="@+id/btnNotify"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Show Notification"
        android:textSize="24sp"
        android:textStyle="bold" />

</LinearLayout>
```

AndroidManifest.xml

```
<?xml version="1.0" encoding="utf-8"?>
<manifest xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:tools="http://schemas.android.com/tools">
    <uses-permission android:name="android.permission.POST_NOTIFICATIONS" />
    <application
        android:allowBackup="true"
        android:dataExtractionRules="@xml/data_extraction_rules"
        android:fullBackupContent="@xml/backup_rules"
        android:icon="@mipmap/ic_launcher"
        android:label="@string/app_name"
        android:roundIcon="@mipmap/ic_launcher_round"
        android:supportsRtl="true"
        android:theme="@style/Theme.M2Pract8"
        tools:targetApi="31">
        <activity
            android:name=".MainActivity"
            android:exported="true">
            <intent-filter>
                <action android:name="android.intent.action.MAIN" />

                <category android:name="android.intent.category.LAUNCHER" />
            </intent-filter>
        </activity>
    </application>

</manifest>
```

MainActivity.kt

```
package com.example.m2pract8

import android.Manifest
```

```
import android.app.NotificationChannel
import android.app.NotificationManager
import android.os.Build
import android.os.Bundle
import android.widget.Button
import androidx.appcompat.app.AppCompatActivity
import androidx.core.app.ActivityCompat
import androidx.core.app.NotificationCompat
import androidx.core.app.NotificationManagerCompat

class MainActivity : AppCompatActivity() {

    private val channelId = "my channel"

    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContentView(R.layout.activity_main)

        createChannel()

        if (Build.VERSION.SDK_INT >= 33) {
            ActivityCompat.requestPermissions(
                this,
                arrayOf(Manifest.permission.POST_NOTIFICATIONS),
                1
            )
        }

        findViewById<Button>(R.id.btnNotify).setOnClickListener {
            showNotification()
        }
    }

    private fun createChannel() {
        if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.O) {
            val channel = NotificationChannel(
                channelId,
                "My Notifications",
                NotificationManager.IMPORTANCE_DEFAULT
            )

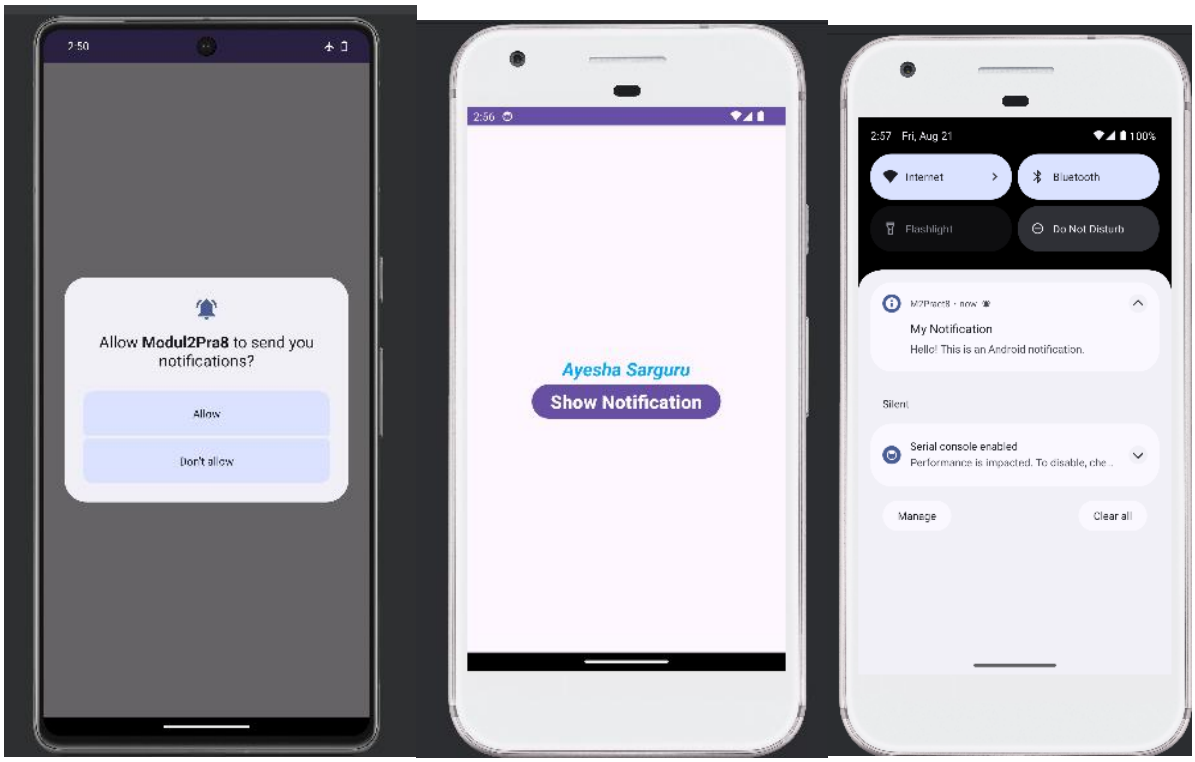
            val manager =
                getSystemService(NotificationManager::class.java)

            manager.createNotificationChannel(channel)
        }
    }

    private fun showNotification() {
        val notification = NotificationCompat.Builder(this, channelId)
            .setSmallIcon(android.R.drawable.ic_dialog_info)
            .setContentTitle("My Notification")
            .setContentText("Hello! This is an Android notification.")
            .setPriority(NotificationCompat.PRIORITY_DEFAULT)
            .setAutoCancel(true)
            .build()

        NotificationManagerCompat.from(this).notify(1, notification)
    }
}
```

Output:


2:50
Allow Modul2Pra8 to send you notifications?
Allow
Don't allow
2:56
Ayesha Sarguru
Show Notification
2:57 Fri, Aug 21
100%
Internet
Bluetooth
Flashlight
Do Not Disturb
My Notification
Hello! This is an Android notification.
Silent
Serial console enabled
Manage
Clear all

Practical 9

Aim: Develop an Android application implementing Static and Dynamic Broadcast Receivers.

A BroadcastReceiver responds to broadcast messages sent by Android or by applications. A static receiver is declared in the manifest where permitted, while a dynamic receiver is registered in code during runtime. Receivers can respond to relevant system or application events, subject to Android's background and security restrictions.

activity_main.xml

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    xmlns:tools="http://schemas.android.com/tools"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:gravity="center"
    android:orientation="vertical"
    tools:context=".MainActivity">

    <TextView
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Ayesha Sarguru"
        android:textColor="#03A9F4"
        android:textSize="24sp"
        android:textStyle="bold|italic" />

    <Button
        android:id="@+id/btnNotify"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Show Notification"
        android:textSize="24sp"
        android:textStyle="bold" />

</LinearLayout>
```

AndroidManifest.xml

```
<?xml version="1.0" encoding="utf-8"?>
<manifest xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:tools="http://schemas.android.com/tools">

    <application
        android:allowBackup="true"
        android:dataExtractionRules="@xml/data_extraction_rules"
        android:fullBackupContent="@xml/backup_rules"
        android:icon="@mipmap/ic_launcher"
        android:label="@string/app_name"
        android:roundIcon="@mipmap/ic_launcher_round"
        android:supportsRtl="true"
        android:theme="@style/Theme.M2Pract9"
        tools:targetApi="31">
        <activity
            android:name=".MainActivity"
            android:exported="true">
            <intent-filter>
                <action android:name="android.intent.action.MAIN" />

                <category android:name="android.intent.category.LAUNCHER" />
            </intent-filter>
        </activity>
        <receiver
            android:name=".MyReceiver"
            android:exported="false">

            <intent-filter>
                <action android:name="MY_BROADCAST" />
            </intent-filter>

        </receiver>
    </application>
```

```
</manifest>
```

MyReceiver.kt

```
package com.example.m2pract9

import android.content.BroadcastReceiver
import android.content.Context
import android.content.Intent
import android.widget.Toast

class MyReceiver : BroadcastReceiver() {

    override fun onReceive(context: Context, intent: Intent) {
        Toast.makeText(
            context,
            "Static Receiver Received",
            Toast.LENGTH_SHORT
        ).show()
    }
}
```

MainActivity.kt

```
package com.example.m2pract9

import android.content.BroadcastReceiver
import android.content.Context
import android.content.Intent
import android.content.IntentFilter
import android.os.Bundle
import android.widget.Button
import android.widget.Toast
import androidx.appcompat.app.AppCompatActivity
import androidx.core.content.ContextCompat

class MainActivity : AppCompatActivity() {

    private val receiver = object : BroadcastReceiver() {
        override fun onReceive(context: Context, intent: Intent) {
            Toast.makeText(
                context,
                "Dynamic Receiver Received",
                Toast.LENGTH_SHORT
            ).show()
        }
    }

    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContentView(R.layout.activity_main)

        val filter = IntentFilter("MY_BROADCAST")
        ContextCompat.registerReceiver(
            this,
            receiver,
            filter,
            ContextCompat.RECEIVER_EXPORTED
        )

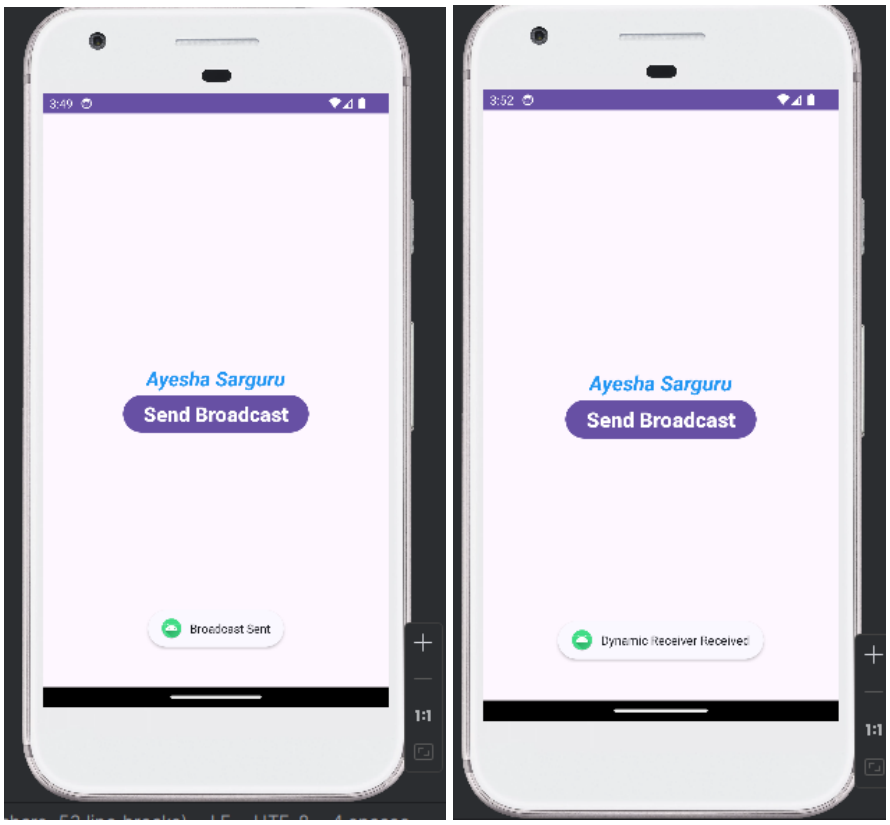
        findViewById<Button>(R.id.btnBroadcast).setOnClickListener {
            val intent = Intent("MY_BROADCAST")
            intent.setPackage(packageName)
            sendBroadcast(intent)

            Toast.makeText(
                this,
                "Broadcast Sent",
                Toast.LENGTH_SHORT
            ).show()
        }
    }

    override fun onDestroy() {
        unregisterReceiver(receiver)
        super.onDestroy()
    }
```

```
}
```

Output:


3:49
Ayesha Sarguru
Send Broadcast
Broadcast Sent
1:1
3:52
Ayesha Sarguru
Send Broadcast
Dynamic Receiver Received
1:1

Practical 10

Aim: Develop an Android application implementing SQLite database CRUD operations.

SQLite is a lightweight relational database available for storing structured data locally on an Android device. CRUD stands for Create, Read, Update, and Delete, the four basic data operations. SQL queries are used to insert, retrieve, modify, and remove records from database tables.

activity_main.xml

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:orientation="vertical"
    android:padding="20dp">

    <EditText
        android:id="@+id/etId"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:hint="Enter ID"
        android:inputType="number" />

    <EditText
        android:id="@+id/etName"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:hint="Enter Name" />

    <Button
        android:id="@+id/btnInsert"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:text="Insert" />

    <Button
        android:id="@+id/btnView"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:text="View" />

    <Button
        android:id="@+id/btnUpdate"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:text="Update" />

    <Button
        android:id="@+id/btnDelete"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:text="Delete" />

    <TextView
        android:id="@+id/tvResult"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:padding="10dp"
        android:textSize="18sp" />

</LinearLayout>
```

MainActivity.kt

```
package com.example.practical10m2

import android.os.Bundle
import android.widget.Button
import android.widget.EditText
import android.widget.TextView
import android.widget.Toast
import androidx.appcompat.app.AppCompatActivity
```

```
class MainActivity : AppCompatActivity() {

    lateinit var db: DBHelper

    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)

        setContentView(R.layout.activity main)

        db = DBHelper(this)

        val id = findViewById<EditText>(R.id.etId)
        val name = findViewById<EditText>(R.id.etName)
        val result = findViewById<TextView>(R.id.tvResult)

        findViewById<Button>(R.id.btnInsert).setOnClickListener {
            if (db.insertData(
                    id.text.toString().toInt(),
                    name.text.toString()
                )
            ) {
                Toast.makeText(
                    this,
                    "Inserted",
                    Toast.LENGTH_SHORT
                ).show()
            }
        }

        findViewById<Button>(R.id.btnView).setOnClickListener {
            result.text = db.getData()
        }

        findViewById<Button>(R.id.btnUpdate).setOnClickListener {
            if (db.updateData(
                    id.text.toString().toInt(),
                    name.text.toString()
                )
            ) {
                Toast.makeText(
                    this,
                    "Updated",
                    Toast.LENGTH_SHORT
                ).show()
            }
        }

        findViewById<Button>(R.id.btnDelete).setOnClickListener {
            if (db.deleteData(id.text.toString().toInt())) {
                Toast.makeText(this, "Deleted", Toast.LENGTH_SHORT).show()
            }
        }
    }
}
```

DBHelper.kt

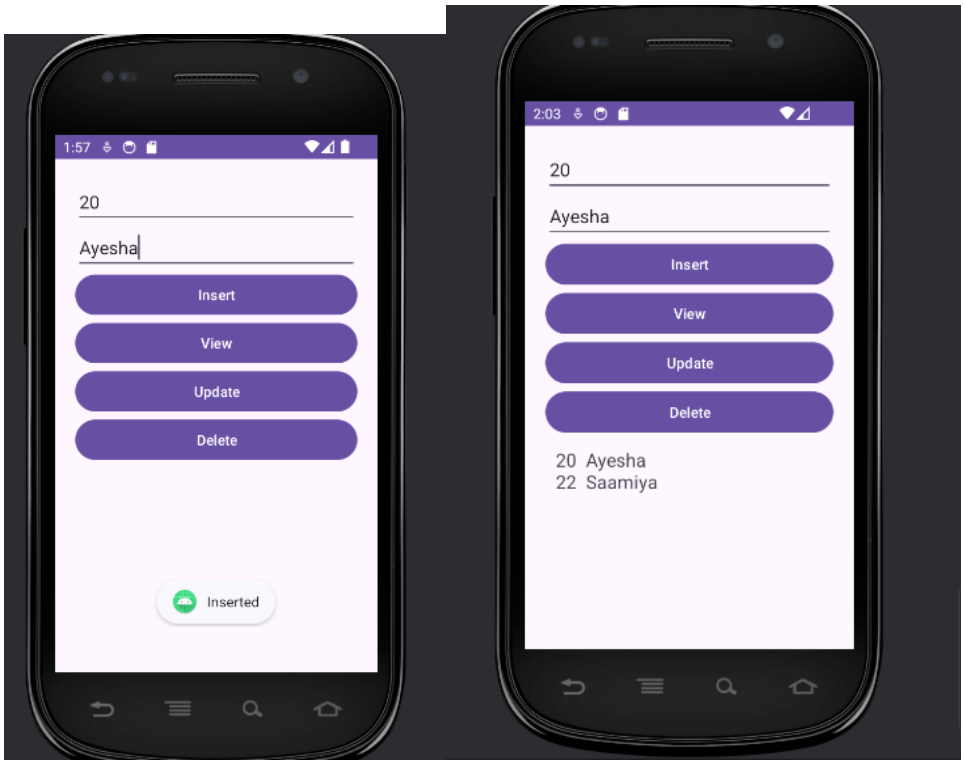
```
package com.example.practical10m2
import android.content.ContentValues
import android.content.Context
import android.database.sqlite.SQLiteDatabase
import android.database.sqlite.SQLiteOpenHelper

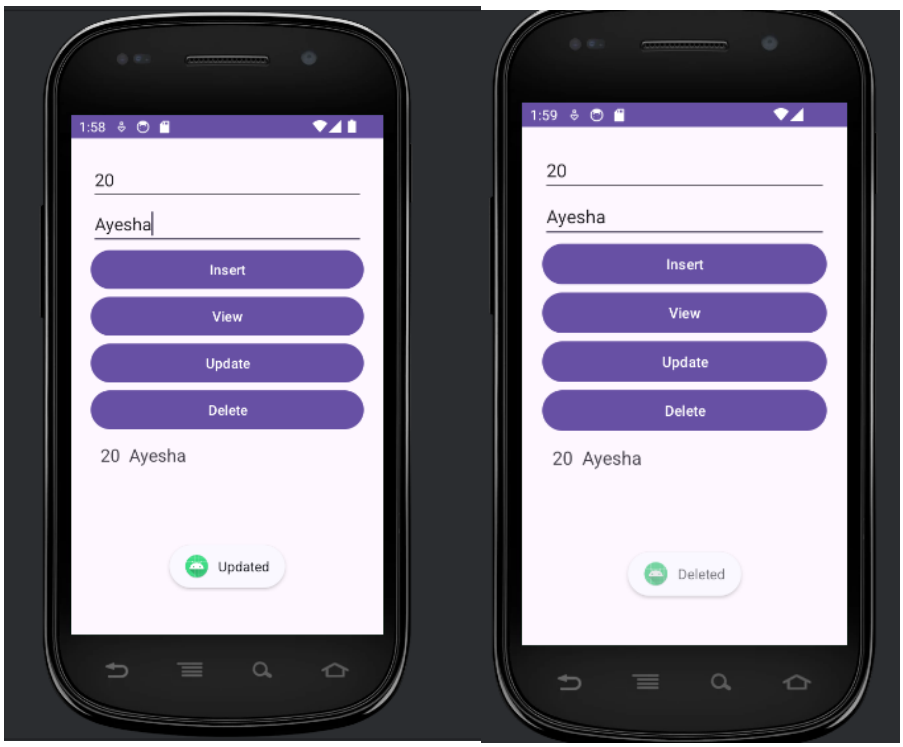
class DBHelper (context:Context) :
    SQLiteOpenHelper(context, "StudentDB", null, 1){
    override fun onCreate(db: SQLiteDatabase) {
        db.execSQL(
            "CREATE TABLE student(id INTEGER PRIMARY KEY, name TEXT )"
        )
    }
    override fun onUpgrade(
        db: SQLiteDatabase,
        oldVersion: Int,
        newVersion: Int
    ){
        db.execSQL("DROP TABLE IF EXISTS student")
        onCreate(db)
    }
```

```
    fun insertData(id: Int, name: String) : Boolean {
        val values = ContentValues()
        values.put("id", id)
        values.put("name", name)
        return writableDatabase.insert("student", null, values) != -1L
    }
    fun getData(): String{
        val cursor = readableDatabase.rawQuery("SELECT * FROM student", null)
        val result = StringBuilder()
        while (cursor.moveToNext()) {
            result.append("${cursor.getInt(0)}  ${cursor.getString(1)}\n")
        }
        cursor.close()
        return result.toString()
    }

    fun updateData(id: Int, name: String) : Boolean {
        val values = ContentValues()
        values.put("name", name)
        return writableDatabase.update("student"20, values, "id=?", arrayOf(id.toString())) > 0
    }
    fun deleteData(id: Int): Boolean {
        return writableDatabase.delete(
            "student",
            "id=?",
            arrayOf(id.toString())
        ) > 0
    }
}
```

Output




1:58
20
Ayesha
Insert
View
Update
Delete
20 Ayesha
Updated
1:59
20
Ayesha
Insert
View
Update
Delete
20 Ayesha
Deleted

Practical 11

Aim: Develop an Android application integrating SQLite with ListView or RecyclerView.

Database records can be retrieved from SQLite and displayed in a ListView or RecyclerView. An adapter binds the retrieved records to the interface, allowing users to view data in an organised format. The displayed list should be refreshed when records are inserted, updated, or deleted.

activity_main.xml

```
<?xml version="1.0" encoding="utf-8"?>

<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout width="match parent"
    android:layout height="match parent"
    android:orientation="vertical"
    android:padding="20dp">

    <EditText
        android:id="@+id/etName"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:hint="Enter Student Name" />

    <Button
        android:id="@+id/btnInsert"
        android:layout width="match parent"
        android:layout_height="wrap_content"
        android:text="Add Student" />

    <ListView
        android:id="@+id/listView"
        android:layout_width="match_parent"
        android:layout_height="match_parent" />

</LinearLayout>
```

DBHelper.kt

```
package com.example.m2pract11

import android.content.ContentValues
import android.content.Context
import android.database.sqlite.SQLiteDatabase
import android.database.sqlite.SQLiteOpenHelper

class DBHelper(context: Context) :
    SQLiteOpenHelper(context, "studentDB", null, 1) {

    override fun onCreate(db: SQLiteDatabase) {
        db.execSQL(
            "CREATE TABLE student(id INTEGER PRIMARY KEY AUTOINCREMENT, name TEXT)"
        )
    }

    override fun onUpgrade(
        db: SQLiteDatabase,
        oldVersion: Int,
        newVersion: Int
    ) {
        db.execSQL("DROP TABLE IF EXISTS student")
        onCreate(db)
    }

    fun insertData(name: String) {
        val values = ContentValues()
        values.put("name", name)
        writableDatabase.insert("student", null, values)
    }

    fun getData(): ArrayList<String> {
        val list = ArrayList<String>()

        val cursor = readableDatabase.rawQuery(
            "SELECT * FROM student",
```

```
            null
        )

        while (cursor.moveToNext()) {
            list.add(cursor.getString(1))
        }

        cursor.close()
        return list
    }
}
```

MainActivity.kt

```
package com.example.m2pract11

import android.os.Bundle
import android.widget.ArrayAdapter
import android.widget.Button
import android.widget.EditText
import android.widget.ListView
import androidx.appcompat.app.AppCompatActivity
import com.example.m2pract11.DBHelper

class MainActivity : AppCompatActivity() {

    lateinit var db: DBHelper
    lateinit var adapter: ArrayAdapter<String>
    val list = ArrayList<String>()

    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContentView(R.layout.activity_main)

        db = DBHelper(this)

        val name = findViewById<EditText>(R.id.etName)
        val button = findViewById<Button>(R.id.btnInsert)
        val listView = findViewById<ListView>(R.id.listView)

        list.addAll(db.getData())

        adapter = ArrayAdapter(
            this,
            android.R.layout.simple_list_item_1,
            list
        )

        listView.adapter = adapter

        button.setOnClickListener {

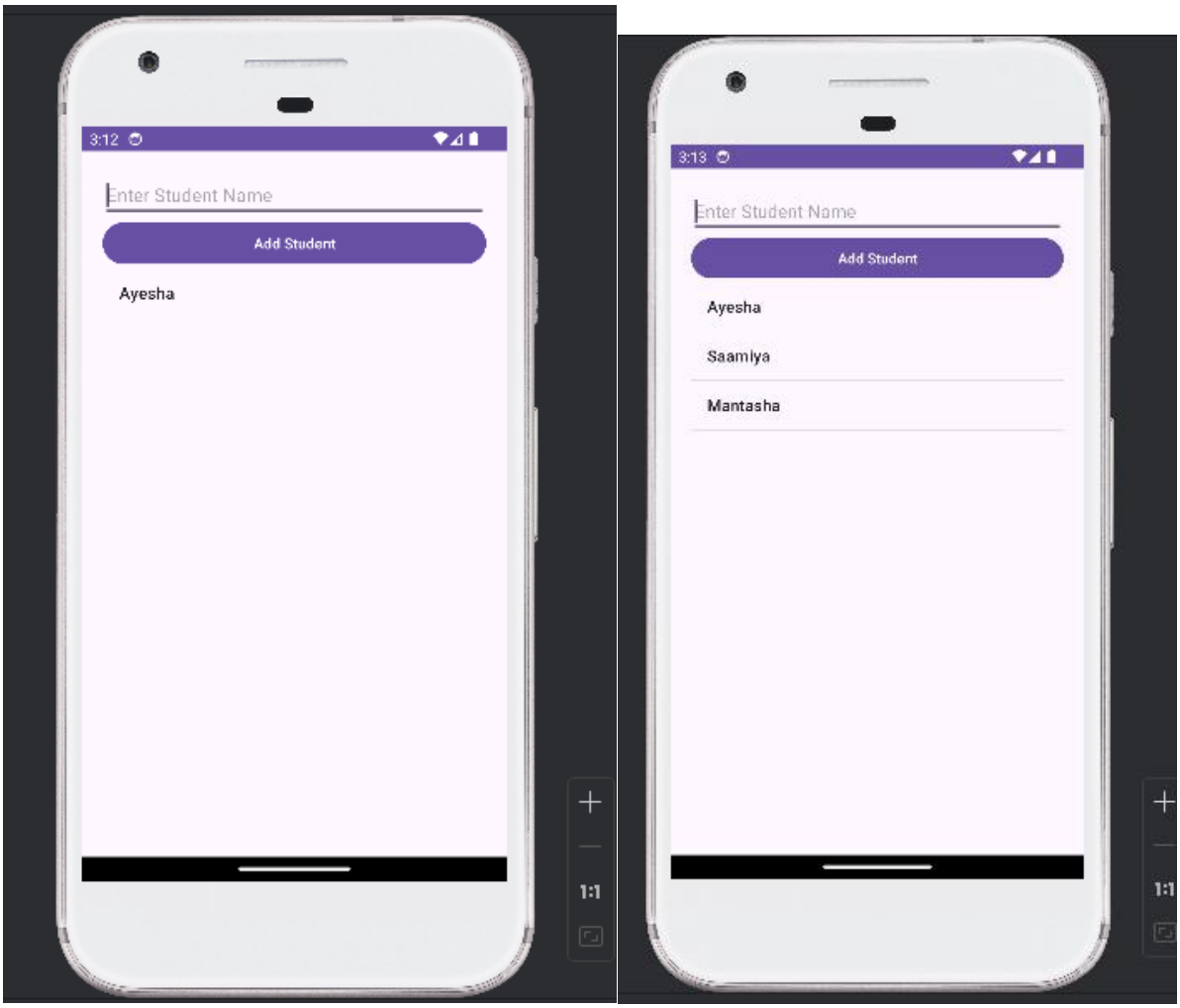
            db.insertData(name.text.toString())

            list.clear()
            list.addAll(db.getData())

            adapter.notifyDataSetChanged()

            name.text.clear()
        }
    }
}
```

Output


3:12
Enter Student Name
Add Student
Ayesha
3:13
Enter Student Name
Add Student
Ayesha
Saamiya
Mantasha

Practical 12

Aim: Develop an Android application to fetch and parse JSON data from a web service and display it in UI components.

JSON (JavaScript Object Notation) is a lightweight format commonly used to exchange data between an application and a web service. The application sends a network request, receives a JSON response, and parses its objects or arrays into usable values. Network operations should be performed away from the main UI thread, with errors handled appropriately.

AndroidManifest.xml

```
<?xml version="1.0" encoding="utf-8"?>
<manifest xmlns:android="http://schemas.android.com/apk/res/android">

    <uses-permission android:name="android.permission.INTERNET" />

    <application
        android:allowBackup="true"
        android:label="Pract12"
        android:supportsRtl="true"
        android:theme="@style/Theme.Pract12">

        <activity
            android:name=".MainActivity"
            android:exported="true">
            <intent-filter>
                <action android:name="android.intent.action.MAIN" />
                <category android:name="android.intent.category.LAUNCHER" />
            </intent-filter>
        </activity>

    </application>
</manifest>
```

activity_main.xml

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout
    xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:orientation="vertical"
    android:background="#FFFFFF"
    android:padding="16dp">

    <ListView
        android:id="@+id/listView"
        android:layout width="match parent"
        android:layout_height="match_parent"
        android:divider="#DDDDDD"
        android:dividerHeight="1dp" />

</LinearLayout>
```

MainActivity.kt

```
package com.example.pract12

import android.os.Bundle
import android.view.Gravity
import android.widget.ArrayAdapter
import android.widget.ListView
import android.widget.TextView
import android.widget.Toast
import androidx.appcompat.app.AppCompatActivity
import org.json.JSONArray
import java.net.HttpURLConnection
import java.net.URL
import kotlin.concurrent.thread

class MainActivity : AppCompatActivity() {
```

```
    private lateinit var listView: ListView

    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContentView(R.layout.activity_main)

        listView = findViewById(R.id.listView)


        val userList = arrayListOf(
            "Ayesha Sarguru \ ayeshasarguru@gmail.com "
        )

        val adapter = ArrayAdapter(
            this,
            android.R.layout.simple_list_item_1,
            userList
        )

        listView.adapter = adapter

        // Fetch JSON data from the web service
        thread {
            var connection: HttpURLConnection? = null

            try {
                connection = URL(
                    "https://jsonplaceholder.typicode.com/users"
                ).openConnection() as HttpURLConnection

                connection.requestMethod = "GET"
                connection.connectTimeout = 10000
                connection.readTimeout = 10000

                if (connection.responseCode == HttpURLConnection.HTTP_OK) {

                    val response = connection.inputStream
                        .bufferedReader()
                        .use { it.readText() }

                    val jsonArray = JSONArray(response)

                    val fetchedUsers = ArrayList<String>()

                    for (i in 0 until jsonArray.length()) {
                        val user = jsonArray.getJSONObject(i)

                        val name = user.getString("name")
                        val email = user.getString("email")

                        fetchedUsers.add("$name\n$email")
                    }

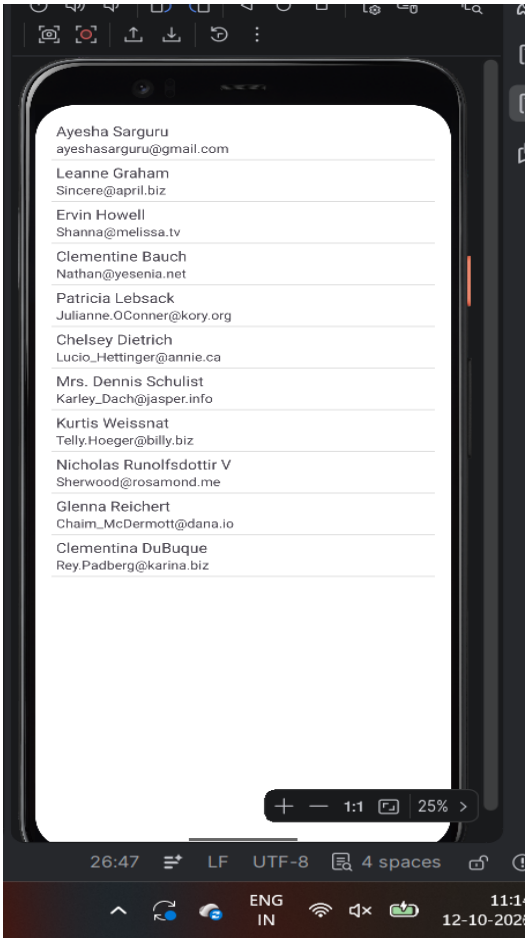
                    runOnUiThread {
                        userList.addAll(fetchedUsers)
                        adapter.notifyDataSetChanged()
                    }

                } else {
                    runOnUiThread {
                        Toast.makeText(
                            this,
                            "Unable to fetch data",
                            Toast.LENGTH_SHORT
                        ).show()
                    }
                }

            } catch (e: Exception) {
                runOnUiThread {
                    Toast.makeText(
                        this,
                        "Error: ${e.message}",
                        Toast.LENGTH_LONG
                    ).show()
                }
            } finally {
                connection?.disconnect()
            }
```

```
            }
        }
}
```

Output:


Ayesha Sarguru
ayeshasarguru@gmail.com
Leanne Graham
Sincere@april.biz
Ervin Howell
Shanna@melissa.tv
Clementine Bauch
Nathan@yesenia.net
Patricia Lebsack
Julianne.OConner@kory.org
Chelsey Dietrich
Lucio_Hettinger@annie.ca
Mrs. Dennis Schulist
Karley_Dach@jasper.info
Kurtis Weissnat
Telly.Hoeger@billy.biz
Nicholas Runolfsdottir V
Sherwood@rosamond.me
Glenna Reichert
Chaim_McDermott@dana.io
Clementina DuBuque
Rey.Padberg@karina.biz
1:1
25%
26:47
LF
UTF-8
4 spaces
ENG
IN

Practical 13

Aim: Develop an Android application demonstrating Threads and Handlers for background processing.

A thread is a path of execution that can perform work independently of the main thread. Android uses the main thread to handle UI events and drawing, so lengthy tasks should not block it. A Handler can schedule messages or tasks to be processed by a particular thread's message queue.

Activity_main.xml

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    xmlns:tools="http://schemas.android.com/tools"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:gravity="center"
    android:orientation="vertical"
    android:padding="20dp"
    tools:context=".MainActivity">

    <TextView
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Ayesha Sarguru - 20"
        android:textColor="#03A9F4"
        android:textSize="20sp"
        android:textStyle="bold|italic" />

    <Button
        android:id="@+id/btnStart"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:gravity="center"
        android:text="Start Processing"
        android:textSize="20sp"
        android:textStyle="bold" />

    <TextView
        android:id="@+id/tvResult"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:gravity="center"
        android:padding="20dp"
        android:text="Click Start Processing"
        android:textSize="20sp"
        android:textStyle="bold" />

</LinearLayout>
```

MainActivity.kt

```
package com.example.pract13

import android.os.Bundle
import android.os.Handler
import android.os.Looper
import android.widget.Button
import android.widget.TextView
import androidx.appcompat.app.AppCompatActivity

class MainActivity : AppCompatActivity() {

    private lateinit var handler: Handler
    private lateinit var textView: TextView

    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)

        setContentView(R.layout.activity_main)

        val button = findViewById<Button>(R.id.btnStart)
```

```
        textView = findViewById(R.id.tvResult)

        // Handler associated with the main UI thread
        handler = Handler(Looper.getMainLooper())

        button.setOnClickListener {
            textView.text = "Processing started..."

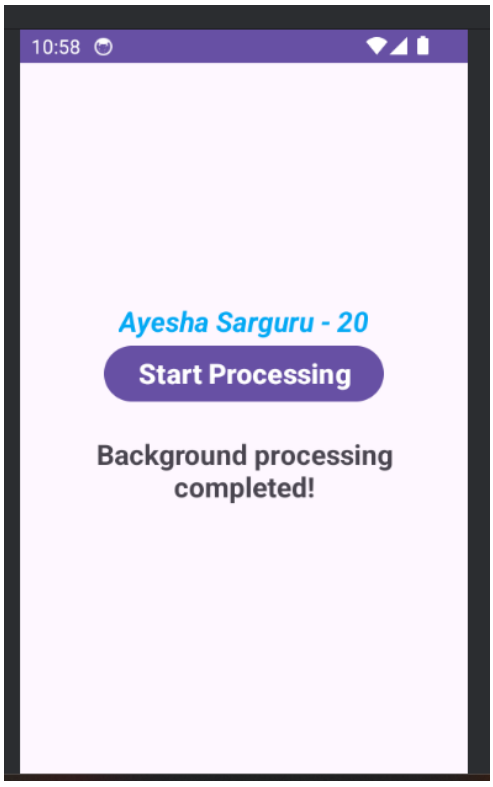
            Thread {
                for (i in 1..10)
                {
                    Thread.sleep(500)

                    handler.post {
                        textView.text = "Processing: $i / 10"
                    }
                }

                handler.post {
                    textView.text = "Background processing completed!"
                }
            }.start()
        }
    }

                }
```

Output:



Practical 14

Aim: Develop an Android application implementing Media APIs and Telephone APIs.

Android provides media APIs for working with audio, video, and other media features. Telephone-related APIs can provide access to supported telephony information or actions, depending on device capability and permissions. Applications should check feature availability and request the necessary permissions before using restricted functions.

Activity_main.xml

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    xmlns:tools="http://schemas.android.com/tools"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:orientation="vertical"
    android:padding="20dp"
    tools:context=".MainActivity">

    <TextView
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:gravity="center"
        android:text="Ayesha Sarguru"
        android:textSize="20sp" />

    <TextView
        android:id="@+id/textView"
        android:layout width="match parent"
        android:layout_height="wrap_content"
        android:gravity="center"
        android:text="Media and Telephone API's"
        android:textSize="22sp"
        android:padding="10dp"/>

    <Button
        android:id="@+id/btnPlay"
        android:layout_width="match_parent"
        android:layout height="wrap content"
        android:text="Play Audio" />

    <Button
        android:id="@+id/btnStop"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:text="Stop Audio" />
    <EditText
        android:id="@+id/etPhone"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:hint="Enter Phone Number"
        android:inputType="phone"/>

    <Button
        android:id="@+id/btnDial"
        android:layout width="match parent"
        android:layout_height="wrap_content"
        android:text="Open Dialer" />

    <TextView
        android:id="@+id/tvInfo"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:padding="20dp"
        android:textSize="18sp" />

</LinearLayout>
```

Mainactivity.kt

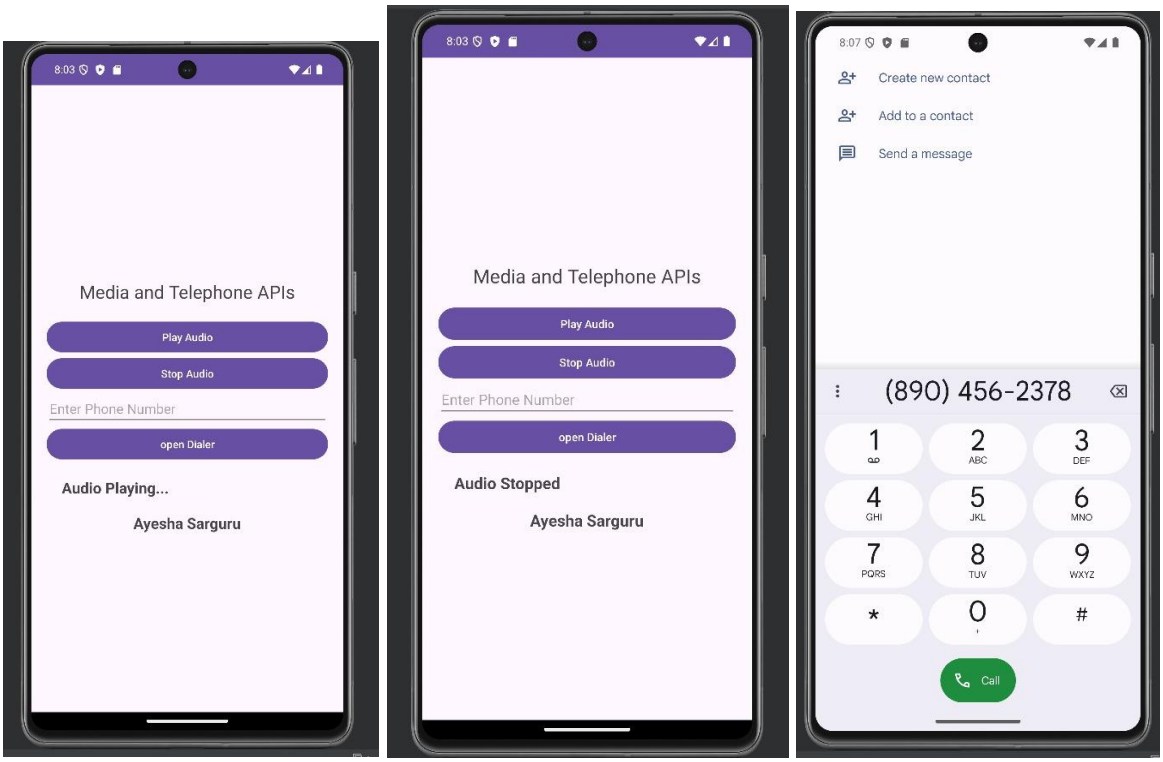
```
package com.example.m2pract14
```

```
import android.net.Uri
import android.content.Intent
import android.content.Context
import android.media.MediaPlayer
import android.telephony.TelephonyManager
import android.os.Bundle
import android.widget.Button
import android.widget.EditText
import android.widget.TextView
import androidx.activity.enableEdgeToEdge
import androidx.appcompat.app.AppCompatActivity
import androidx.core.view.ViewCompat
import androidx.core.view.WindowInsetsCompat

class MainActivity : AppCompatActivity() {
    lateinit var mediaPlayer: MediaPlayer
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        enableEdgeToEdge()
        setContentView(R.layout.activity_main)
        val btnPlay = findViewById<Button>(R.id.btnPlay)
        val btnStop = findViewById<Button>(R.id.btnStop)
        val btnDial = findViewById<Button>(R.id.btnDial)
        val etPhone = findViewById<EditText>(R.id.etPhone)
        val tvInfo = findViewById<TextView>(R.id.tvInfo)
        //Media Player
        mediaPlayer = MediaPlayer.create(
            this, R.raw.sample_audio
        )
        btnPlay.setOnClickListener {
            if (!mediaPlayer.isPlaying) {
                mediaPlayer.start()
                tvInfo.text = "Audio Started"
            }
        }
        btnStop.setOnClickListener {
            if (mediaPlayer.isPlaying) {
                mediaPlayer.pause()
                tvInfo.text = "Audio Stopped"
            }
        }
        //Telephone API - Open Dialer
        btnDial.setOnClickListener {
            val phoneNumber = etPhone.text.toString()
            val intent = Intent (
                Intent.ACTION_DIAL,
                Uri.parse("tel:$phoneNumber")
            )
            startActivity(intent)
        }
        //Telephone API
        val telephonyManager = getSystemService(Context.TELEPHONY_SERVICE)
                as TelephonyManager
        tvInfo.text = "Phone Type : ${telephonyManager.phoneType}"
    }

    override fun onDestroy() {
        mediaPlayer.release()
        super.onDestroy()
    }
}
```

Output:


8:03
Media and Telephone APIs
Play Audio
Stop Audio
Enter Phone Number
open Dialer
Audio Playing...
Ayesha Sarguru
8:03
Media and Telephone APIs
Play Audio
Stop Audio
Enter Phone Number
open Dialer
Audio Stopped
Ayesha Sarguru
8:07
Create new contact
Add to a contact
Send a message
(890) 456-2378
1
2 ABC
3 DEF
4 GHI
5 JKL
6 MNO
7 PQRS
8 TUV
9 WXYZ
*
0 +
#
Call

Practical 15

Aim: Develop an Android application implementing Runtime Permissions and security handling in AndroidManifest.xml.

Android permissions control access to protected resources such as location, camera, microphone, and phone features. Dangerous permissions must be requested from the user at runtime on applicable Android versions. The application should explain why access is needed, check permission results, and handle denial safely.

AndroidManifest.xml

```
<?xml version="1.0" encoding="utf-8"?>
<manifest xmlns:android="http://schemas.android.com/apk/res/android"
    package="com.example.runtimepermissionapp">

    <!-- Declare the permission required by the application -->
    <uses-permission android:name="android.permission.CAMERA" />

    <application
        android:allowBackup="true"
        android:icon="@mipmap/ic launcher"
        android:label="Runtime Permission App"
        android:roundIcon="@mipmap/ic_launcher_round"
        android:supportsRtl="true"
        android:theme="@style/Theme.AppCompat.Light.DarkActionBar">

        <activity
            android:name=".MainActivity"
            android:exported="true">
            <intent-filter>
                <action android:name="intent.action.MAIN" />
                <category android:name="intent.category.LAUNCHER" />
            </intent-filter>
        </activity>
    </application>
</manifest>
```

activity_main.xml

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:gravity="center"
    android:orientation="vertical"
    android:padding="24dp">

    <Button
        android:id="@+id/btnCheckPermission"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:text="Request Camera Permission"
        android:textSize="16sp" />

</LinearLayout>
```

Mainactivity.kt

```
package com.example.runtimepermissionapp

import android.Manifest
import android.content.Intent
import android.content.pm.PackageManager
import android.os.Bundle
import android.provider.MediaStore
import android.widget.Button
import android.widget.Toast
import androidx.activity.result.contract.ActivityResultContracts
import androidx.appcompat.app.AppCompatActivity
import androidx.core.content.ContextCompat

class MainActivity : AppCompatActivity() {

    // Launcher to handle camera permission request
    private val requestPermissionLauncher =
```

```
        registerForActivityResult(ActivityResultContracts.RequestPermission()) { isGranted ->
            if (isGranted) {
                Toast.makeText(this, "Camera Permission Granted!", Toast.LENGTH_SHORT).show()
                openCamera()
            } else {
                Toast.makeText(this, "Camera Permission Denied!", Toast.LENGTH_SHORT).show()
            }
        }

    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContentView(R.layout.activity main)

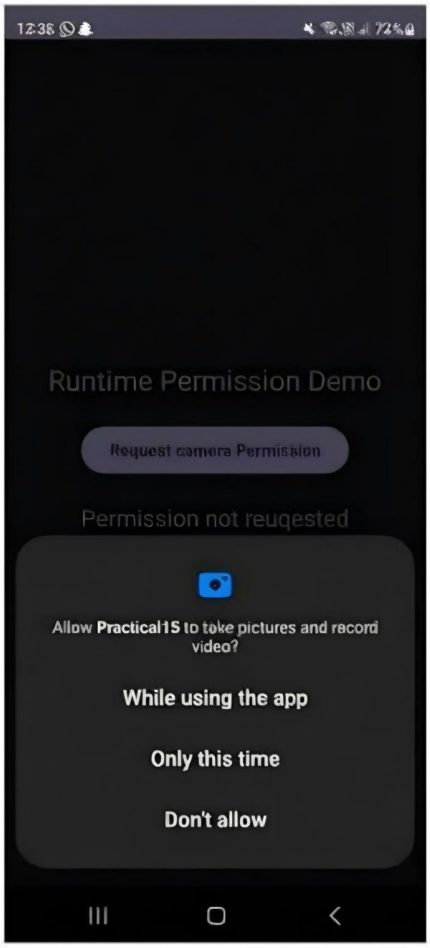
        val btnCheckPermission = findViewById<Button>(R.id.btnCheckPermission)

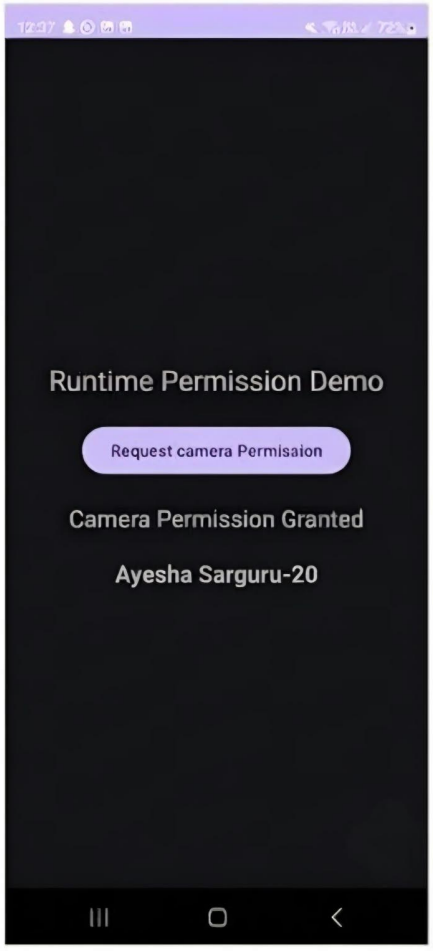
        btnCheckPermission.setOnClickListener {
            checkAndRequestCameraPermission()
        }
    }

    private fun checkAndRequestCameraPermission() {
        when {
            ContextCompat.checkSelfPermission(this, Manifest.permission.CAMERA) == PackageManager.PERMISSION_GRANTED
-> {
                Toast.makeText(this, "Camera Permission already granted for Ayesha Sarguru ",
Toast.LENGTH_SHORT).show()
                openCamera()
            }
            else -> {
                // Request the camera permission directly
                requestPermissionLauncher.launch(Manifest.permission.CAMERA)
            }
        }
    }

    private fun openCamera() {
        val cameraIntent = Intent(MediaStore.ACTION_IMAGE_CAPTURE)
        if (cameraIntent.resolveActivity(packageManager) != null) {
            startActivity(cameraIntent)
        } else {
            Toast.makeText(this, "No camera app found on device", Toast.LENGTH_SHORT).show()
        }
    }
}
```

Output:


Runtime Permission Demo
Request camera Permission
Permission not reuqested
Allow Practical15 to take pictures and record video?
While using the app
Only this time
Don't allow


Runtime Permission Demo
Request camera Permisaion
Camera Permission Granted
Ayesha Sarguru-20